

目录

1 文档范围与源码基线	4
2 系统组成与职责边界	5
3 Core：运行角色、命名与启动	6
4 Core：普通调用与 Targeted 调用	7
5 Core：协作规划与选择事务	8
6 Core：对象传输、连续流与流式调用	9
7 Core：能力、并发、持久状态与观测	10
8 Core：身份验证与请求级保密	11
9 Core：授权版本与权威	12
10 Core：在线授予权限	13
11 Core：在线撤回权限	14
12 Core：版本发现与传播	15
13 Core：授权变化的运行影响	16
14 Repo：架构与对象模型	17
15 Repo：持久化、写入与原子可见性	18
16 Repo：大制品传输与可恢复会话	19
17 Repo：目录、副本、保留与恢复	20
18 DI：应用入口与当前实现层次	21
19 DI：输入、模型图、候选与放置	22
20 DI：计划封装、执行授权与制品	23

21 DI: 原生角色执行与依赖数据流	24
22 DI: 逐 token 生成、KV 与会话延续	25
23 DI: 部署、扩展与管理接口	26
24 UAV: 服务容器、接口与操作界面	27
25 UAV: 飞控适配与操作权限	28
26 UAV: 巡逻任务、补偿与事件协作	29
27 UAV: 视频、遥测流与录像	30
28 UAV: 目标检测与多视角识别	31
29 跨模块协作的完整路径	32
30 构建、配置与部署边界	33
31 实现状态与验证口径	34
32 源码索引: Core 与授权	35
33 源码索引: Repo 与 DI	36
34 源码索引: UAV 与交付	37
35 文档维护与后续目标变更	38
36 API 阅读方法与覆盖边界	39
37 Core API: 容器与角色生命周期	39
38 Core API: 请求发起与 Targeted 调用	40
39 Core API: Provider 注册与作用域回收	41
40 Core API: 延迟规划与协作提交	43
41 Core API: 调用流的事件和终态	44

42 Core API: 命名对象与连续流	45
43 Core API: 在线授权与版本状态	46
44 Core API: 执行租约、观测与并发	47
45 Repo API: 对象与精确 Data packet	48
46 Repo API: 制品上传与提交	49
47 Repo API: 下载、验证与传输状态机	50
48 Repo API: 目录、副本与缓存控制	51
49 DI API: Python 应用入口与原生句柄	52
50 DI API: 模型适配与角色放置扩展	54
51 DI API: 准备、发布后再绑定与封存	55
52 DI API: Provider 注册、授权与执行	56
53 DI API: 生成、KV 与会话延续	57
54 UAV API: 飞控边界与操作权限	58
55 UAV API: 任务、补偿与终端报告	59
56 UAV API: 视频管线、遥测与录像	60
57 UAV API: 检测与多视角识别扩展	61
58 API 扩展与跨模块调用检查	62

1 文档范围与源码基线

NDNSF、NDNSF-UAV、NDNSF-DI、NDNSF-Repo 完整设计视图

本文覆盖四个模块的职责、主要接口、控制与数据流程、安全、状态、恢复、配置、部署和实现边界。按子系统解释设计，不逐行转录源代码。代码标识、协议字段和文件名保留原名，其余叙述使用中文。

版本 R1；当前设计与目标设计使用完全一致的技术正文。新增 API 契约与声明参考，目标仍未增加相对当前代码的新行为。

基线 2026-09-07，Experimental 分支。采样时提交为 e9fe33994a6c；逐文件摘要、工作树差异与本地源码快照见 source-baseline.json。

事实口径 主要流程核对源码；已有接口与已接通流程分别说明。历史测试仅作为检索入口，不冒充本轮结果。

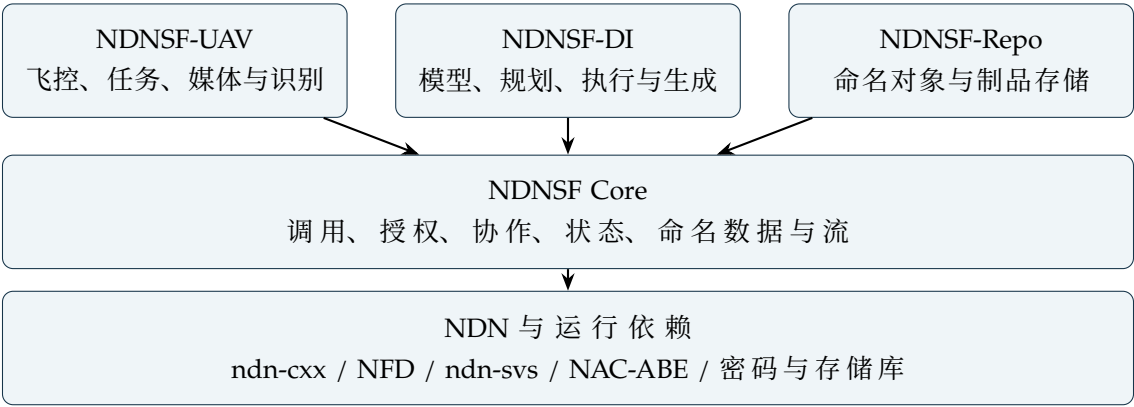
版本维护 本目录完整纳入 Git。两份正文独立保存；修改目标设计不自动修改当前设计或代码。

内容组	覆盖内容
整体与 Core	所有者关系、调用、协作、数据传输、并发与状态。
授权与安全	请求级保密、版本、授予/撤回、状态传播与更新影响。
Repo	对象/制品、持久化、传输、目录、副本与恢复。
DI	应用接口、规划、安全、原生执行、生成/KV、部署与扩展。
UAV	容器、飞控、任务、操作权限、视频、录像及多视角识别。
交付与证据	跨模块流程、运行环境、实现状态、源码索引及维护。

关键现状 已有 C++ Provider 执行组件不代表纯 C++ 请求全链路已经完成；当前原生 DI dispatch 仍返回 NATIVE_REQUEST_PIPELINE_NOT_READY。权限 grant 的局部取钥优化也不代表所有无关节点的运行状态完全不受影响。

2 系统组成与职责边界

Core 提供通用机制；应用模块拥有领域语义



UAV 可调用 Repo 保存录像，也可调用分析服务；DI 可调用 Repo 发布、复用或获取模型制品。这些是应用间依赖，不意味着 UAV 飞行状态、模型图或 Repo 目录语义进入 Core。[C1, R1, D1, U1]

模块	代码位置与所有权
Core	ndn-service-framework/；通用协议与 C++ runtime。
UAV	NDNSF-UAV-APP/；drone、ground-station、shared 及工具/配置。
DI	NDNSF-DistributedInference/；C++ 角色执行与迁移中的原生请求端、Python 应用和策略层。
Repo	NDNSF-DistributedRepo/；C++ 库/服务与 Python 制品编排。
Python 绑定	pythonWrapper/；Core 绑定与应用辅助层，不等价于全部逻辑均在 C++。

边界规则 Core 可以传递、认证和保存应用定义的 payload，但不解释 waypoint、tensor、model shard、codec 或 object class 的领域含义。

3 Core: 运行角色、命名与启动

身份、服务与进程容器相互关联，但不是同一个对象

ServiceController 维护授权策略和撤权状态，提供签名权限响应及 PolicyStatus，持有 NAC-ABE Authority。

ServiceUser 发布请求、收集 ACK、选择参与者、发送 Selection、处理 Response；维护请求、token、版本与超时状态。

ServiceProvider 注册服务和 ACK/执行处理器；响应候选发现、验证选择、运行任务、发布结果。

ServiceContainer 同一进程组合 Controller/User/Provider 角色和本地服务，管理启动、停止及生命周期 hook。

LocalServiceRegistry 提供可信进程内调用；不是网络调用模式，不增加线上的权限属性或服务选择语义。[C1, C2]

3.1 名字与消息

服务通过 NDN 名称寻址；Request、ACK、Selection、Response 由 Core 的类型和命名 helper 编码。Provider 身份、服务名、requestId 与消息类别共同约束匹配；具体文法以 NDNSFMessages.* 和 utils.* 为准，不由应用拼装新的旁路协议。

通用服务可由多个 Provider 提供；应用需要指向一个物理节点时可使用 Provider 专属服务名，或在已经完成 token bootstrap 后使用 Targeted。名字可达、服务被发现和当前获准执行是不同条件。

3.2 启动与安全准备

加载配置、身份/证书、信任模式，建立 Face 和服务注册，再获取权限和 ABE 材料。构造期异步获取 DKEY 不能阻塞整个应用运行；但 bootstrap-pending 不构成执行授权。Controller 的启动含独立 Face 的公参可达性探测，避免仅以内进程命中宣称 NFD 路径可用。[C2, S3]

配置集中在 ConfigManager、PolicyParser 及应用配置文件；服务容器只负责组合，硬件连接、模型加载和数据库路径由各应用模块决定。

4 Core: 普通调用与 Targeted 调用

先完成发现与选择，再执行；已知 Provider 可使用受保护的短路径

1. User 为新 invocation 分配 requestId、token 和时间预算，发布 Request。
2. 可服务且具有相应权限的 Provider 返回 ACK，携带 Provider token、能力和应用自定义报价。
3. ACK 窗口结束后，User 从候选中选择一个或多个 Provider。策略可为 first-responding、random、all-selected 或自定义。
4. User 发布绑定实际候选与 token 的 Selection；未选中节点不能据 Request 自行开始执行。
5. Provider 验证选择、权限与请求绑定，调用处理器；User 验证返回的签名、密钥和 requestId，再触发结果回调。[C1, C3]



4.1 Targeted 的适用条件

已知 Provider 且已通过正常认证流程取得一次性 token pair 后，后续调用可省略 ACK/Selection。权限、Provider 绑定、请求/响应认证和 token 防重放仍然存在。token 缺失、过期或耗尽时，应走相应 bootstrap/错误处理，不能退化成无认证的直发。[C3]

4.2 失败的归属

Nack、没有可用 ACK、负 ACK、选择失败、执行错误、响应验证失败与超时是不同边界。发布成功不等于执行成功；普通 ACK 也不天然表示独占资源预留。

5 Core: 协作规划与选择事务

不可变 ACK 快照与一次计划提交把应用规划接入同一 invocation

DEFERRED 协作先发起通用请求, 再收集 ACK_CLOSED 快照; 应用根据快照决定角色、参与者、依赖和密钥作用域。规划完成后调用同一次协作的 commit_plan。PREPLANNED 保留请求前已知计划的兼容入口, 两者不能被混成一次动态重开 ACK 窗口。[C3, D2]

5.1 Core 负责的契约

角色标识、精确参与者绑定、依赖名称/作用域、请求与 attempt 关联、选择消息分发及终态处理, 由通用协作契约承载。模型切分可行性、UAV 任务分区或 Repo 副本评分由上层判断。

GenericSelectionTxnStore 保存认证后的选择上下文与应用提交 blob; OpaqueSelectionParticipant 提供 prepare、onCommitted、onAborted。事务关联 Provider boot epoch、token、可选 lease、截止时间、payload digest 和存储 key epoch; 重放已提交记录也检查当前运行代次。[C4]

5.2 带资源预留的选择

R1 的 DI reservation 是独立协商能力, 不能赋予所有普通 ACK 相同语义。启用时 Provider 先完成有界资源预留, 再给 positive ACK; 选择后提交/准备, 未选中或迟到候选收到 NOT_SELECTED。资源最终由本地 lease 到期收回。

选中 source role 本地就绪即可运行; 有依赖的角色还要获得其直接前驱的认证数据。不把旧兼容 ReadySet/ExecutionActivate 接口解释为所有当前路径都需要的全局启动屏障。[D2]

完成边界 计划提交、Provider 接受、资源准备完成、角色执行完成、终端结果被接受是不同状态; 文档和应用状态显示应分别记录。

6 Core：对象传输、连续流与流式调用

按数据语义选择入口，不仅按字节数选择

数据形态	当前接口与用途
完整命名对象	<code>publishLargeNamed</code> 、 <code>fetchLarge</code> 、 <code>fetchLargeExact</code> ；确定名字和版本的模型、 <code>tensor bundle</code> 、 <code>manifest</code> 、录像文件。
连续发布流	<code>StreamPublisher</code> 、 <code>PredictiveStreamSubscriber</code> 、 <code>StreamInfo/StreamChunk</code> ；不断产生的媒体、遥测与状态。
单次调用的事件流	<code>RequestServiceStreaming</code> 、 <code>StreamedResponseWriter</code> ；一个 <code>invocation</code> 内的有界事件、取消和终止，如生成 <code>token</code> 。

6.1 完整对象

请求中可携带小型引用；实际字节按精确 `Data` 名获取并验证。大响应的请求级保密路径按段 `AEAD`，必须具有相应的请求密钥/绑定。`Repo` 增加持久存储和副本语义，不建立另一套无命名大 `payload` 通道。[C5]

6.2 连续流

通用流管理会话身份、`cursor`、名称映射、序列缺口、重排、重复、保留窗口、预测请求与背压。当前 `Stream` 还提供对不透明字节的有界 `FEC` 修复能力；媒体帧类型、编码参数和画面呈现仍归 `UAV`。流恢复必须受期限/保留范围限制，不能无限阻塞后续交付。[C6]

6.3 调用事件流

事件绑定 `service`、`request`、`Provider`、`ControllerVersion` 和 `generation` 等上下文。有界队列、事件保留和重试、`transcript/terminal` 校验、取消与 `deadline` 共同控制生命周期。终态只被接受一次，晚到事件不得复活已经结束的请求。普通调用与 `Targeted` 均须满足各自授权前提。[C7]

选择例子 直播使用连续流；已完成录像使用命名对象；一个推理请求逐 `token` 输出使用调用事件流。三者可以组合，但不能互相冒充。

7 Core：能力、并发、持久状态与观测

公共状态表达事实，上层策略解释业务意义

能力与发现 通用 capability/runtime hint 表达可用性、负载、队列、资源和 drain 状态；应用字段以带 schema 的 payload 附着。可达不代表可以接收新任务。

准入与租约 ProviderAdmissionLeaseTable 和 ExecutionLease 维护有期限的准入/执行状态；request、service、Provider、资源绑定及消费/释放检查防止跨请求复用。

服务生命周期 注册可使用作用域句柄，关闭后按注册代次阻断晚到回调。一个 Provider host 可共享物理资源 lease 状态，避免多个服务各自重复预订同一槽位。

操作状态 通用状态区分排队、执行、等待输入、完成、失败、取消和过期；结果以类型化 metadata 或 DataProductReference 表达。[C2, C8]

7.1 线程与终态

Face 的 I/O 线程、ACK/handler 工作池以及应用后台线程承担不同工作。pending 请求和资源表由相应锁/代次保护；取消和关闭要清理绑定，不能仅删除 GUI 条目。Core 的事件回调、请求 deadline 与应用长生命周期任务仍分开拥有。

7.2 持久化不等于恢复完成

ControllerGenerationStore 管理持久授权版本；RuntimeStatusStore 可选保存签名状态并重新验证；选择事务有自己的 WAL/存储密钥。DI 的请求 journal、UAV 的 mission ledger、Repo 的数据库是上层持久状态，不能用一个文件存在代替全部恢复握手。[C4, S3, S12]

7.3 可观测性

NetworkTelemetry、TimelineTrace 和应用 diagnostics 提供事件与时序信息。应区分排队/网络/验证/装配/计算/呈现时间；成功标记、源码 hash 和原始运行目录需要关联。单项测试或某个统计计数不自动构成端到端性能或安全资格结论。

8 Core：身份验证与请求级保密

签名验证、服务授权和内容保密解决不同问题

8.1 身份与信任

证书与 Trust Schema 确认 Data 的签名身份及名称是否符合信任规则。Controller 的授权策略决定身份能否使用或提供服务；签名有效本身不等于拥有服务权限。加密证书与签名证书可分开配置，但须满足同一身份及运行要求；NAC-ABE 使用的加密证书需要兼容其密钥封装。[C2, C5]

8.2 默认 V2 请求级保密路径

1. 初始 Request 的加密发现信息面向获授权的 Provider 类别，使用户不必预先知道具体执行者。
2. 选择后，为本次调用准备独立的输入/响应密钥；输入作为 User 签名的精确命名 Data，在输入密钥下加密。
3. SelectionKeyEnvelope 使用选中 Provider 的 RSA-OAEP 封装传递 K_input 与 K_response，绑定接收者证书及请求上下文。
4. 响应使用 AES-GCM；AAD 包含 service、requestId、ControllerVersion、attempt、证书摘要等绑定。分段或事件使用不同 nonce，nonce registry 拒绝复用。
5. 终态/失效路径释放或清零请求秘密；大响应仍需逐段验证及请求绑定，不能退回共享服务响应钥匙。[C5, S7]

8.3 ABE 与其他密钥的范围

ABE 公参和 DKEY 用于属性策略下的授权/保密；请求 AES 密钥用于本次输入/输出；HybridMessageCrypto 仍用于其他服务级、协作或 token 场景。DI artifact grant、UAV 视频内容钥匙和操作权 lease 是各自应用契约，不与这些字段混成一个 epoch。

8.4 证书和公参可达性

安全 bootstrap 需要实际名称路由、Controller/Authority 可达和签名验证。NFD 前缀注册、forwarding hint 或本地缓存只影响数据如何到达，不能取代身份和内容验证；失败要按证书、公参、权限、状态或内容层定位。

9 Core：授权版本与权威

版本推进、密钥轮换和权限变化是三个不同判断

对象	当前语义
ControllerVersion	两个非零 uint64：generation timestamp 与 epoch；先比较 generation，再比较 epoch。Controller 的同一全局版本用于各服务的 PolicyStatus。[S1, S2]
PolicyStatusData	按服务返回的签名状态：ControllerVersion、有效期、策略摘要、ABE 公参名字和摘要、Controller 证书、相关撤权记录。[S2]
ABE generation	以实际公参 Data name + digest 区分。grant-only 保持不变；withdrawal 轮换。节点据此判断是否清空共享 NAC 缓存。[S3, S6]
PermissionResponse	面向目标身份的权限表，携带 policy epoch。节点替换本地记录并比较权限内容，决定是否需要目标 DKEY 刷新。[S5]
DKEY	当前 KP-ABE 实现按身份持有完整属性策略的解密密钥，并非每次新增一个服务就附加一个独立属性密钥。[S3, S4]

9.1 不能仅从字段推导的结论

advanceAuthorizationEpoch() 同时设置 m_policyEpoch 与 m_requiredKeyEpoch 为新 epoch；这不表示每次调用都进行了 ABE 全局轮换。运行时另行比较公参身份。[S3]

getPolicyStatus() 对每个服务装入相同的 ControllerVersion，再过滤服务级撤权记录；身份级和证书级撤权也被加入状态。当前结构不是独立的“每身份权限修订号”。[S2]

该函数的 policyDigest 来源于配置文件 SHA-256；不能把它当作完整在线 grant/revoke 变更集摘要，也不能仅凭其不变就判断某节点不受影响。[S2]

9.2 权威边界

经认证的业务消息所携带的新版本只能触发状态拉取。安装状态必须验证 Controller 签名、服务、有效期及版本绑定；旧版本被拒绝，同版本但内容冲突也被拒绝。状态与刷新协调器先在副本上验证，再一起安装。[S6, S8, S9]

不能混用的两个结论 “ABE 参数未变”意味着不必全局重发 ABE 密钥；它本身不证明该服务所有在途请求仍满足新的授权状态。

10 Core：在线授予权限

保留 ABE 参数，仅替换目标身份的完整策略

1. 应用或控制入口调用 `ServiceController::grant(identity,service,attribute)`。User 使用 `/PERMISSION/...`，Provider 使用 `/SERVICE/...`。
2. 验证目标及 Controller 状态；若此前有未完成的撤权轮换，先恢复该轮换，失败则拒绝本次 `grant`。证书撤销不可由 `grant` 绕过。
3. 更新目标身份的服务授权和属性集合，移除匹配的服务撤权；显式重新授权可解除该身份的身份级撤权。
4. 替换目标身份的完整有效属性策略，重建查找表，再推进全局授权 epoch。
5. Authority 在该身份下一次请求时生成替换 DKEY。普通 `grant-only` 不调用全局 ABE 参数轮换，也不遍历其他身份重发策略。[S4]

10.1 目标身份如何获得新增权限

权限发现需要应用层续期或显式 `fetchPermissionsFromController()`。仅知道新 `PolicyStatus`，并不自动等价于获取新增权限；User/Provider 对权限响应和状态响应分别处理。

`applyPermissionResponse()` 比较更新前后的权限内容：User 比较服务集合；Provider 比较 Provider/service 记录集合。仅内容确实改变时，标记相关服务等待 DKEY 更新。随后安装状态时，若 ABE 参数未变且存在待刷新标记，则执行 DKEY-only refresh。[S5, S6]

同一 `ControllerVersion` 的多个服务响应合并为一次身份 DKEY 更新；状态先到、权限后到的反向顺序也由同版本安装路径处理。相同代次刷新期间，旧密钥保留到替换密钥安装。[S6, S10]

10.2 其他身份会发生什么

没有权限内容变化且 ABE 公参身份不变时，无关身份不会仅因为更高 epoch 而请求替换 DKEY。不过，Provider 接受已安装状态的版本推进后会重新拉取自身权限表；运行状态失效也仍然可能发生。详见“Core：授权变化的运行影响”一节。[S5–S7]

本流程的限定范围 上述不轮换结论限定于普通 `grant-only`。若 `grant` 入口先恢复了此前失败的撤权轮换，此次调用包含那次轮换的影响，不能归类为纯目标身份更新。

11 Core: 在线撤回权限

撤回权限，同时切换后续加密使用的 ABE 代次

1. `revoke(target)` 支持身份、证书或特定身份的服务权限撤回；先验证目标并处理已有 `pending rotation`。
2. 将撤权目标加入集合，持久推进 `ControllerVersion`。持久化失败时恢复本次内存修改。
3. 轮换 `Authority` 的 ABE 主密钥/公共参数代次；公参版本由 `generation` 与 `epoch` 派生。
4. 对已知身份遍历剩余有效属性：先移除旧策略；无剩余权限则不再设置策略，否则设置完整过滤后策略。客户端后续获取对应 DKEY。
5. 后续 `PolicyStatus` 返回当前版本、当前 ABE 公参身份和相关撤权记录；`User/Provider` 拉取、验证并安装后，触发授权与缓存处理。[S3, S8]

11.1 失败处理顺序

授权 `epoch` 已持久化而 ABE 轮换抛错时，不回退到旧版本。撤权保留，设置 `m_abeRotationPending`，调用返回失败。下一次 `revoke`（包括同目标重试）或 `grant` 先执行恢复；恢复会预留更高 `epoch`，再轮换，以免改写已经发布的同版本状态。[S3]

这是 `Controller` 本地失败处理，不表示离线或尚未更新的远端节点已经获知撤权。`Controller` 成功、远端发现状态、客户端重新取钥、受保护操作拒绝旧授权，是不同观察点。

11.2 密码学与运行时边界

新代次下加密的数据不能用旧代次 DKEY 正常解密。已经交付的旧明文、旧密钥或旧代次密文不因此被追溯收回。运行中的操作还依赖本地版本门、请求绑定与失效处理，不能仅用“已换钥”描述全部撤权效果。

在节点安装不同公参身份后，会清理本地共享 `NAC Consumer` 缓存、获取新 DKEY，并清理/刷新 `Producer` 的公参状态。这个共享 ABE 范围大于单个服务的本地 `token` 范围。[S6, S7, S10]

证据边界 本文件描述代码顺序和拒绝路径；没有给出撤权全网生效时延、网络分区下的完成保证或本轮运行测试结论。

12 Core：版本发现与传播

当前通过 NDN 拉取状态；没有独立的 Controller SVS epoch 公告

触发方式	当前行为
权限获取/更新	权限路径触发相应 PolicyStatus 获取;新增权限依赖应用续期或显式 refetch。 [S5]
经认证的新版本 hint	协调器检查 hint 是否可信、有效且更新；合并正在进行的拉取，按目标版本确认签名状态。[S9]
临近过期刷新	默认按当前状态到期时间安排，携带当前精确版本；旧精确版本请求本身不能发现新 epoch。[S11]
周期重新检查	NDNSF_POLICY_REVALIDATION_PERIOD_MS 大于 0 时使用固定周期，并发出不指定版本的 fresh 状态请求。默认值 0，关闭此周期模式。[S11]
可选状态恢复	恢复持久状态先重新验证，再向 Controller 做在线确认；这不是对离线最新状态的证明。[S12]

12.1 Interest 与 Data 契约

不指定版本的请求前缀为

<controller>/NDNSF/POLICY-STATUS/<service>

客户端设置 CanBePrefix=true 和 MustBeFresh=true；Controller 用当前精确版本名返回签名 Data, FreshnessPeriod=0。指定版本时客户端关闭前缀匹配；Controller 若已不提供该版本则不回答。客户端验证 Data 名字、内部版本和期望版本的一致性。[S8]

12.2 SVS 在当前设计中的位置

ServiceController 的当前实现未建立 epoch 专用 SVS publisher/subscriber。框架其他业务使用 SVS, 不代表授权更新已经具备相同公告路径。本文件及首版目标设计均不把“SVS 通知后拉取状态”方案写成已实现能力。

时效边界 仅推进 Controller epoch 不会主动唤醒所有节点。周期模式、显式拉取和消息 hint 的触发条件决定何时发现变化；本轮没有测量发现或完成时间，也不承诺所有节点立即同时切换。

13 Core：授权变化的运行影响

本地已实现部分区分，但尚非“无关节点完全无感”

设 Provider A 获得服务 S 的新权限，Provider B 的权限内容没有变化，ABE 参数仍为 P，Controller 版本由 V 变为 V+1。以下描述 B 或 User **实际接受服务 S 的新状态**之后的代码分支，而非自动广播后的全网结果。

状态或动作	当前结果
B 的 ABE DKEY	比较权限内容无变化，且公参身份未变时，不执行 DKEY-only refresh。Authority 也没有替换 B 的策略。[S4–S6]
B 的权限表	Provider 从已有状态推进版本时，触发自身权限 refetch；LocalMock 跳过这一网络动作。[S7]
服务 S 的本地状态	新版本安装调用 invalidateControllerScopedCaches，并未先判断变更是否仅属于另一个身份。[S6]
User 的在途调用	同服务 stream 被拒绝；普通调用保留 pending/deadline，但清除请求秘密、绑定、token 等，让终态/超时路径继续负责结束。[S7]
Provider 的在途状态	清理匹配服务的 pending 请求、token、stream 绑定和相关发布状态；不是按 grant 的目标身份过滤。[S7]
其他服务 T	本次服务 S 的失效按服务匹配。但各服务状态携带同一全局 ControllerVersion；若稍后接受 T 的更新状态，T 也会进入自己的失效路径。[S2, S6, S7]

13.1 当前设计的准确结论

可以说：普通 grant-only 只替换目标身份的 ABE 策略；无关身份不因 epoch 变化而重新取 DKEY。
不能说：普通 grant-only 只影响目标 Provider，其他节点无须拉取或更新任何状态、所有在途调用不受影响。这超过当前运行状态失效逻辑所支持的结论。

13.2 目标 R0 的边界

本版尚未采纳更新范围优化或 SVS 通知。因此目标 R0 保留本页全部现状。后续若修改目标设计，应明确影响判定、变更认证、版本兼容、离线补齐和在途请求处置，再讨论实现；当前代码与验收门不随文档标题自动改变。

14 Repo：架构与对象模型

命名对象存储与领域无关；C++ 库和 Python 编排各有边界

RepoCore 不依赖 NDNSF 网络调用的进程内存存储门面；提供 put/get/list/remove、manifest、Data packet、容量和目录操作。

RepoNode 将 RepoCore 挂接到 ServiceProvider 或 LocalServiceRegistry，注册存储操作服务，管理请求响应及 operation 状态。

RepoClient 提供直接、进程内和网络 helper；Python 的 py_repoclient 进一步提供副本、目录和制品会话编排。

存储后端 普通对象/packet 的持久后端与大制品的 PayloadStore/MetadataStore 接口分别承载不同物化方式。[R1–R4]

数据对象	关键身份与含义
RepoObjectManifest	objectName、objectType、SHA-256、size、segmentCount、replicationFactor、replicaNodes、policyEpoch 及生命周期 metadata。
已签名 Data packet	按原始 NDN 名称与 wire bytes 保存；保持发布者身份，不能因进入 Repo 就改成 Repo 内容签名。
ArtifactReference	不可变制品身份：逻辑名、内容摘要、大小、root manifest 名称、发布者与策略 epoch；不是只有路径。
目录条目	逻辑对象、manifest 摘要、来源 Repo、状态、目录版本及副本位置；不等于全部段字节。

14.1 控制面与数据面

默认服务前缀为 /NDNSF/DistributedRepo。CAPABILITY、INSERT、MANIFEST、INVENTORY、STATUS、CACHE_STATUS、CATALOG 系列和 DELETE 等属于控制操作；具体 C++/Python 可用操作以各自注册表为准，不能假定两个入口完全对等。数据面仍通过原始精确名称获取分段 Data，forwarding hint 可帮助定位副本。[R2, R7]

通用性 Repo 不解释模型层、视频帧和张量。payload 是否加密、如何解码和何时可以用于业务，由发布者、消费者及应用权限机制决定。

15 Repo：持久化、写入与原子可见性

先提交权威存储，再更新缓存；收到字节不等于发布完成

15.1 普通对象与 packet 存储

部署中的普通 Repo-node 对象路径使用 SQLite 权威存储，叠加按字节预算限制的 LRU 热点缓存。写入先提交数据库，再纳入缓存；读取 miss 回到数据库；删除/覆盖先完成权威事务。超过缓存预算的条目旁路缓存；缓存和计数随进程消失，持久对象不会因缓存清空而被视为丢失。in-memory backend 是测试替身，不应当成受支持的持久部署模式。[R3]

15.2 大制品后端

PayloadStore 管理分范围写入、verified ranges、flush/finalize/abort；MetadataStore 管理生命周期事件。RepositoryStoreFacade 结合后端所有权 lease，防止多个 owner 无约束地修改同一存储目录。FilesystemArtifactStore 和本地 Python artifact backend 支持文件/staging 物化，与普通 SQLite 对象 API 不能混称为一个实现。[R4]

15.3 网络写入流程

1. 发布者确定精确制品身份和 manifest，公开供选中 Repo 拉取的数据源。
2. 通过 NDNSF 协作发现容量/能力，固定副本选择和 operationId。
3. 各 Repo 获取、校验并持久化数据，产生绑定 operation、artifact 和 Repo 身份的提交 receipt。
4. 客户端验证收到的 receipt 与已提交的选择集合及内容身份一致，满足要求后才返回发布完成。

分段收到、哈希验证、持久提交和对外可见分别记录。副本不足或 receipt 不匹配不能用“部分上传成功”包装成完整发布。[R5, R6]

范围区别 普通存储 helper 的重试/分段适配器与 artifact-manifest-v2 的会话生命周期不是同一层；不能把某一条本地路径通过当作所有网络后端都通过。

16 Repo: 大制品传输与可恢复会话

公开 API 隐藏 packet 编排, 保留精确身份和提交检查

接口	语义
<code>publish_file / publish_file_async</code>	校验源文件身份, 发布到要求的副本数, 返回不可变引用及副本结果。
<code>fetch_file / fetch_file_async</code>	按引用获取并校验文件, 完成后物化到目标位置。
<code>begin_upload → upload_file → commit</code>	显式上传会话, 区分传输与提交。
<code>begin_fetch → transfer → commit</code>	显式获取会话, 区分下载与最终可见文件。
<code>abort(preserve_progress=True)</code>	保留已验证 staging, 允许在相同身份与恢复条件下续传。

16.1 清单、分段与背压

artifact-manifest-v2 将 root manifest、清单页和实际 payload 分开, 避免把海量段列表塞进控制消息。AdaptiveArtifactTransfer 跟踪 in-flight、重传、重复和 verification backlog; 收到段先离开 Interest 窗口, 但在验证/持久化完成前继续施加背压。恢复会话绑定 artifact、root digest、chunk/packet 参数和 lease; 不能把相同文件名当作可续传证明。[R4–R6]

16.2 当前网络入口的限制

CollaborationArtifactApiBackend 承载网络制品发布; 当前 `_NetworkPublishDriver.transfer()` 明确拒绝 TARGETED 整制品发布, 要求 Collaboration。API 中存在 Targeted 控制选项, 不表示每个具体 backend 的所有操作都支持它。[R6]

默认路径按能力与资源边界选择副本, 并验证选中集合对应的持久 receipt。本地单副本 backend 用于本地运行/开发, 不代表已经经历真实 NDN 多副本传输。

恢复原则 只复用经过验证的精确进度; 过期 lease、不同 digest、不同分段参数或错误 operation 绑定应由会话检查拒绝, 不能静默接着写入另一个对象。

17 Repo：目录、副本、保留与恢复

目录负责定位和生命周期；对象字节仍由数据面获取

每个 Repo 有本地目录，Python 控制编排还提供目录交换、查询和修复工具。系统没有把单一集中 NameNode 设为所有对象读取的必要前提；各入口实际是否启用 peer 同步，取决于所选服务和配置，而不是仅因为存在 catalog 类型就自动发生。[R1, R2, R7]

17.1 目录接口

CATALOG_STATUS 返回 Repo 模式、目录 epoch、对象数量和副本接收能力。

SNAPSHOT / DELTA 提供全量恢复或指定 epoch 之后的变化；目录 sequence 不等于 Controller 授权 epoch。

LOOKUP / QUERY 查找对象位置，按类型、class、发布者、状态、标签和时间等 metadata 过滤；C++ 与 Python 注册能力需分别确认。

17.2 副本选择与确认

候选 ACK 中的 freeBytes、mode、负载等用于 placement；选中不代表已经保存。策略需满足复制数量/容量约束，真正成功由验证后的写入 receipt 确认。已知副本位置可优化请求，但仍保留名称、签名、manifest 和内容摘要检查。

17.3 删除、TTL 与修复

删除以 tombstone/目录状态表达，避免旧 AVAILABLE 记录无条件恢复对象。对象可带 ttlMs 与 repairAllowed；过期或不允许修复的对象不应作为新副本修复来源。恢复机制需区分 metadata 恢复、字节验证和实际 replica 完成，不能仅依据目录列表计副本数。[R7]

17.4 应用集成

UAV 的内嵌 Repo 为加密录像和原始 packet 留存提供存储；DI 使用公开 artifact API 传输模型制品。独立 Repo-node、应用内 RepoCore 和 Python 网络制品后端是不同部署形态；文档中的能力应绑定具体入口。

18 DI: 应用入口与当前实现层次

Python 编排仍在；原生执行组件与原生请求入口的完成度不同

ndnsf_distributed_inference.api 当前为受审查的重新导出口，实际实现仍在 app_sdk 等 owner 中。InferenceApplication 管理应用定义/部署；InferenceClient/APPClient 接受请求并提供结果、取消、观察和恢复接口；InferenceProvider 对接实际网络 Provider。导入名字短不代表内部已经没有 Python 控制逻辑。[D1]

18.1 两类应用请求

模型意图请求 以 ModelRef、task、input 和 timeout 提交；AutomaticPlanningCoordinator 在 ACK 窗口关闭后产生图与分割候选、验证 Provider offer、决定计划并提交。

显式预规划请求 使用 RequestableDeployment 或既定 plan；保留兼容 API，不应与无需应用先指定 Provider/role 的延迟规划语义混用。

18.2 C++ 已有的能力

NativeExecutionPlan、NativeProviderSession/Runtime、ProviderRoleWorker、模型 runner、依赖传输、生成协调器及 Provider host 等组件已存在。原生规划、grant、ONNX、tokenizer 和请求生命周期正在 Spec182 中逐步整合。[D3–D6]

18.3 明确未闭合的路径

当前 NativeInferenceClient::request() 会创建有 deadline 的 handle，调度原生操作；dispatchOperation() 仍以 NATIVE_REQUEST_PIPELINE_NOT_READY 结束。因此“纯 C++ 应用已完成默认端到端推理”不是当前可支持的结论。真实 catalog/Repo 输入准备、完整报价消费及请求编排接线仍有未完成项。[D7]

目标 R0 的口径 Spec182 的目标是同一原生实现配可选 Python 外壳；本文件的目标 R0 仍复制当前代码现状，不把这个尚未完成的目标写成已经实现。

19 DI: 输入、模型图、候选与放置

先冻结真实观测，再做模型相关的规划

1. 应用输入经过 adapter 预处理，绑定模型内容/语义摘要与输入类型；大输入使用命名引用。
2. 以 DEFERRED 协作获取不可变 ACK_CLOSED，绑定 request、attempt 和总 deadline。
3. 由固定 adapter 描述模型并检查图，枚举有界、图有效的 split candidates。
4. 从真实 ACK payload 构造 ProviderOfferV3；检查身份、offer 签名、Core 认证来源、模型/图绑定与新鲜度。
5. 放置策略评估候选的完整 role/rank、backend/device、内存、队列和驻留证据；独立验证策略输出，不把策略返回值直接作为执行授权。[D2, D3]

19.1 驻留证据与准备

loaded、assembled、canonical 与 cold 表示不同准备成本。has_model 不能证明 exact residency；加载证据还需匹配进程/boot、runtime、设备、artifact、graph、role 和有效期。具有所需服务能力但尚未加载的 Provider，可以在契约允许时接受带准备的执行；不能通过伪造“已驻留”来获得资格。

19.2 策略扩展

ModelFamilyAdapter 拥有图/切分/物化语义；ModelPlacementStrategy 选择 Provider 与设备。Pre-SplitFirst 系列优先复用合法、精确匹配的候选与驻留证据，再考虑准备成本。策略受候选数量、期限和验证器限制，不能扩大自己的权限或绕过 Controller。

19.3 当前原生迁移

NativeRequestPreparation、NativeOfferAdmission、NativeV3Placement 已有对应组件和检查；当前真实数据来源端口及 requester 主链仍需接线。原生组件的定向测试不能替代模型意图经过网络 ACK 到终端结果的整体验收。[D3, D7]

20 DI：计划封装、执行授权与制品

计划完整性、权限与制品保密分别绑定

计划包含模型/图身份、角色与 rank、Provider assignment、依赖边、设备和执行约束。V3 的 proposal、plan core、Provider grant view 与 selection projection 分开表示不同可见范围；规范编码和摘要用于验证绑定，不能以临时 JSON 字符串或目录顺序代替身份。[D2, D4]

20.1 从决策到选择

1. 验证完整候选与 assignment，确定需要的可信物化和制品。
2. 通过负责该模型的 adapter/assembler 生成合法切片或装配产物。
3. 使用公开 Repo API 发布/确保制品可获取，保留内容与 manifest 身份。
4. 封装不可变计划与授权绑定，向选中 Provider 提交其最小必要投影。
5. Provider 验证 selection、计划、角色、lease、证书/代次与制品绑定后再进入准备/执行。

20.2 原生安全组件

NativeGrantClient/Verifier、NativePlanSealer、NativeProtectedArtifactStore、NativeProtected-Provider 和 ProtectedRuntime 分担 grant、验证、受保护制品和明文生命周期。ProtectedRuntime 的 lease/zeroization 组件用于显式受保护路径；不能因此推导所有普通模型加载或默认 Python 路径都已启用同等的模型制品加密。[D4]

20.3 三个不同的安全问题

Core 的请求保密保护调用输入/输出；DI 的模型/计划身份保证执行的是被指定的内容；受保护 artifact grant 控制特定制品的解密与运行时明文持有。三者必须分别说明，不能用“使用了 ABE”概括所有模型权限。

当前限制 Spec182 的封装、grant 和准备 owner 虽已有实现片段，真实请求端接线与全部验收尚未闭合；接口和类型存在不等于默认网络路径已使用它。

21 DI：原生角色执行与依赖数据流

角色执行在本地就绪后推进，依赖边通过认证命名对象连接

组件	主要职责
NativeExecutionPlan	角色、依赖边、张量与执行身份；NativeProviderAssignment 指定实际 owner。
NativeProviderSession	将计划、assignment、DependencyIo 和 runner factory 组合成会话。
NativeProviderRuntime	维护 runner 注册与 worker，安排角色执行。
ProviderRoleWorker	等待/预取直接输入、运行角色、按依赖边发布输出；管理局部状态和缓存。
NativeProviderHandler	将 Core CollaborationContext 接到原生 session,验证执行绑定、readiness 与授权。

21.1 输入与输出

TensorBundleCodec 定义张量 bundle 编解码；模型 adapter/runner 验证 dtype、shape、名字、图输入/输出及必要的边界转换。DependencyIo 把数据流边映射为精确命名对象，在 producer/consumer/request/attempt/plan 等绑定下获取和发布。同机角色可使用本地 I/O；网络角色通过 NDNSF 的数据传输接口协作。[D5]

21.2 调度与资源

DependencyWaitScheduler/AsyncDataflowRuntime 负责在依赖可用时推进，避免把等待网络的任务无限占住执行线程。CollectiveRuntime、ProviderGroupCoordinator 等组件表达 group/rank 协调；这些类型和执行路径的存在不能自动作为多 GPU、扩展性或吞吐测量结果。

21.3 模型运行器

OnnxRuntimeModelRunner 以及 ONNX/Qwen/YOLO adapter 处理各自的模型格式和运行契约。设备、CUDA/CPU execution provider、runner 身份与 runtime ABI 必须按实际配置核对；不能把允许 GPU 的配置当成实际使用 GPU。输出发布和 terminal role 接受仍由当前执行绑定约束。[D5]

22 DI: 逐 token 生成、KV 与会话延续

不可变模型缓存与带身份的推理状态缓存分开管理

22.1 生成循环

NativeEpochCoordinator 组织每轮生成：读取输入/反馈、校验 lineage、执行角色、抽取 logits、按生成配置采样 token、发布 token/终止事件，并处理取消与截止时间。QwenGenerationSession、GenerationEpochLineage 和 DecodeStateIdentity 记录模型、前缀、position、epoch 及角色状态之间的关系。[D6]

22.2 流与终态

生成事件经 Core 的调用事件流传输；stable text decoding 需要处理 token/字节边界、特殊 token、最终 flush 和不能稳定输出的前缀。EOS、stop sequence、max tokens、失败、取消和 deadline 是不同结束原因。最终结果的归属不能由任意中间角色声明。

22.3 KV 复用

KV 不是普通不可变模型 artifact。可复用状态必须匹配模型内容与语义、adapter、runner、split、tokenizer、精确 prefix/长度、position、层范围、precision/layout、runtime ABI、Requester/安全域及有效期。过期、驱逐、重启或 pin 丢失不能继续作为命中证据。Provider session 提供 host/GPU pause、prefetch、pin、release 等显式状态操作。[D5, D6]

22.4 跨轮对话

ConversationStateBinding 和 NativeConversationCoordinator 用于绑定会话/轮次及状态来源，Python app journal 也保存请求和恢复记录。延续不是重放上一轮的未经验证对象；旧 attempt、晚到结果和已取消状态必须经过相应门禁。

验收边界 当前已有 tokenizer、epoch 和 conversation 组件；Spec182 的 sampling parity、stable epoch emission、journal/continuation 与完整原生 stream 接线仍保留未完成验收项。不把某些部件测试通过写成完整生成路径已迁移。

23 DI: 部署、扩展与管理接口

服务注册、制品准备、执行与管理动作各有生命周期

应用/部署层 app_sdk.application、deployment、controller 和 runtime_journal 管理定义、修订、请求引用、状态与恢复；显式部署可用于预热或兼容 workflow。

Provider 管理 APPProvider/ProviderAdminPort 提供 stage、activate、drain、delete、进度/检查点/输出报告；管理凭据与普通推理请求权限不是同一个授权入口。

原生 host NativeInferenceProvider 通过作用域注册句柄提供服务；共享 ExecutionLease 状态与 target 路由绑定物理槽位，关闭/代次阻断晚到回调。

扩展 SDK registry、loader、bounded executor 和 observer 注册策略/适配器；能力包含分割、放置、调度、准入、缓存、恢复、部署和执行调优等。

23.1 可选后端与工具

维护目录中有 ONNX、Qwen、YOLO adapter，Python 侧还保留 llama 等适配器与兼容入口；模型导出、图检查、GUI、ops 和 experiment tooling 不属于通用 Core。一个 API 导出不表示对应可选库、模型文件或 runtime 当前已安装。[D1, D8]

23.2 持久记录

部署修订、请求/attempt、计划摘要、Provider boot、grant、执行 lease 和结果引用需要一起定位，不能只靠一个 deployment 名称恢复。恢复时应按 owner 的契约重新验证仍有效的运行资源。

23.3 当前迁移状态

原生 Provider host/注册/共享 lease 已有进度记录；Python facade 兼容迁移、maintained caller 迁移及完整请求链仍未全部关闭。文档将 Python 控制路径和 C++ 组件同时列出，避免把 Spec182 的未来单一实现方向覆盖当前仍实际存在的 owner。[D7]

24 UAV：服务容器、接口与操作界面

地面站与无人机都是可同时发起和提供服务的应用进程

UavDroneApp 组合飞控适配、视频、遥测、任务与内嵌 Repo；UavGroundStationApp 组合命令客户端、地图/任务界面、流订阅、记录播放和分析服务。主体代码拆在 Drone/GroundStationServiceContainer 与 Window 的 include 文件中；共享类型和命名位于 shared/。[U1, U2]

服务组	当前名称/后缀及作用
飞行命令	/UAV/MAVLink/Execute；受保护的 MAVLink 执行。
任务与状态	/UAV/Mission/Assign、/UAV/Telemetry/GetStatus；任务分配与遥测。
摄像头	/UAV/Camera/GetFrame、Provider 专属 /UAV/Camera/Video；取帧与开始/停止流。
录像与目录	Provider 专属 /UAV/Camera/Recording/Manifest、/UAV/Camera/Repo/Catalog。
参数与诊断	Provider 专属 Parameters、ParameterEdit、AnalyzeSnapshot 与 Preflight/Checklist。
识别与协作	/UAV/GS/ObjectDetection、/UAV/Incident/Analyze。
操作权管理	GS 的 OperatorAuthority/Lease、Revocation、Audit 服务。

名字可由 UavRuntimeConfig 配置；以上默认值不能代替部署配置和签名身份检查。

24.1 界面与状态模型

GUI 使用 typed TelemetryState、ReadinessState、FlightCommandState、MissionState、MissionProgressState、VideoState、AnalyzeSnapshot 和 dashboard 数据，区分无人机可达性、飞控 readiness、摄像头可用性、任务占用和视频会话。参数浏览/修改、preflight、目录查询和诊断由服务调用支撑，不直接绕过 Core 访问远端设备。[U2, U3]

应用边界 Core 传递 opaque MAVLink/应用 payload；飞行语义、地图与 GUI、媒体选择和检测模型均由 UAV 应用拥有。

25 UAV：飞控适配与操作权限

网络授权、操作权和飞行安全条件是三层独立检查

25.1 飞控数据路径

GS 构造 MAVLink 命令，经 NDNSF 普通认证/token bootstrap 后对目标无人机使用 Targeted。Drone 的后端负责向 mock、UDP 或 serial 飞控路径传递字节并读取 ACK/遥测；mavlink-router 是部署连接方式之一，不是新的 Core 调用协议。[U2]

FlightControllerBackend 定义 sendMavlink、遥测、参数和 mission waypoint 接口。UDP backend 包含连接、帧解析、heartbeat、命令 ACK 和 mission upload 交互，并支持串口配置分支。GPS 信息来自飞控 MAVLink telemetry，而不是把任意本机 GPS 设备直接当作 NDNSF 身份可信来源。

25.2 三层权限与门禁

1. Core 验证 User/Provider 权限、签名、token、ControllerVersion 与请求绑定。
2. UAV 的 OperatorAuthorityLease 关联操作员、目标、范围和有效时间；issuer 提供申请、撤回、审计、持久化和续期逻辑。
3. GroundStation 的 flight/mission safety gate 使用新鲜遥测、readiness、armed/landed 等应用状态判断动作是否允许；lease 不能替代飞控本身的保护。

操作权 issuer 不等于 Core ServiceController；其 lease 修订也不是 Controller epoch。UI 阻止命令、网络拒绝授权和飞控返回 command ACK 必须分别显示。[U2, U3]

25.3 现有支持与限制

mock/SITL、UDP、serial、遥测显示与参数操作为可运行入口；真实飞控、fail-safe、长时间断连恢复和户外设备条件仍需对应运行验证。代码具备硬件接口不表示本轮进行了飞行验证，也不等同于成熟飞控地面站的认证。

26 UAV：巡逻任务、补偿与事件协作

长生命周期 mission 包含多个有限的 NDNSF invocation

26.1 巡逻分配

地面站将 waypoint 集合确定性分区/聚类, 形成任务 plan 与多个 part; 通过共享 /UAV/Mission/Assign 服务发现空闲 Drone。忙碌 Drone 可以发负 ACK 并报告 mission slot 占用; 应用根据候选选择, 而不是用 payload 里的 Provider 名称绕过协商。[U2, U4]

26.2 缺失部分的补偿

patrol task 关联 taskId、attemptId、partId、assignedProvider、deadline、状态和 response digest。某 part 超时或没有有效结果时, 下一次调用只补偿缺失部分; 每次仍有新的 Core requestId/token。任务业务“完成”需要所有必要 part 的有效结果, 而不是单次 invocation 有一个 Response。

26.3 任务与 job 状态

UavMissionSession 维护 planned、starting、active、degraded、compensating、recovering 到完成/取消/失败的任务状态, 并限制历史大小。每个 incident/job 另有 ACK 收集、ACK 关闭、计划提交、selected、evidence-ready、executing、reporting 和 terminal 状态。[U3, U4]

26.4 证据与终端归属

证据使用生产者精确命名 Data, 绑定 mission/incident、digest、时间与身份。终端报告必须匹配当前 attempt、request、planDigest 和选中的 terminal owner; 同一 incident 已接收的终态摘要不会被另一份晚到报告替换。恢复过程需要确认车辆与 stream 绑定, 不能只载入一个旧 ledger 就继续发飞行命令。[U4]

与 Core 的分工 Core 负责每次调用的认证与超时; UAV 负责 part 是否完成、哪些工作需要补偿、跨调用的任务身份以及任务层最终完成。

27 UAV：视频、遥测流与录像

小型控制请求建立会话，高频媒体通过命名 Data 传输

27.1 采集与呈现

VideoPublisher 使用 GStreamer 或兼容管线,支持摄像头/文件/测试来源。UavVideoPipeline 的 frame 结构携带 sessionEpoch、sourceFrameId、captureOriginNs、codec PTS、codecConfigEpoch、keyFrame 与编码字节；有界最新帧队列控制积压。readiness 报告真实来源及失败原因，fallback sample 不能当作真实摄像头采集证据。[U2, U5]

27.2 网络流与自适应

Provider 专属控制服务决定 start/stop；GS 订阅当前 live session，按 cursor/名称与元数据预测、取回和重排 Data。旧会话迟到包不进入当前视频。Core 提供通用窗口、名称映射、恢复与背压；UAV 根据帧类别、码率、画面时效和播放队列做媒体策略，并记录采集、到达、解码与呈现的不同时间点。

27.3 保密与留存

视频会话拥有受保护的 descriptor、密钥/nonce 与 Data 绑定。原始已签名 packet 可以进入 Repo；留存清单记录确切 packet、缺口和会话身份，不通过重新签名抹去原发布者。开始直播与是否开启 recording/retention 是不同控制条件；录像完成后通过 manifest/catalog 发现，按名字获取、验证、解密并播放。[U3, U5]

27.4 数据产品边界

当前具体 Repo-backed 产品主要是相机录像/packet 留存。Mission image、telemetry log、检测事件或报告虽可使用同样的 DataProductReference 模型，不能仅因类型存在就称所有产品均有完整采集和发布链。

时序边界 网络收到、解码完成与实际显示不是同一事件；低延迟或稳定性结论需要真实采集/呈现记录，不能用一个管线启动成功代替。

28 UAV：目标检测与多视角识别

有限识别 job 与长期任务分离，验证输入与结果的实际 owner

28.1 单视角辅助检测

Drone 可调用 GS 的 ObjectDetection 服务，请求只携带 frame/drone/目标类别等紧凑元数据，由 GS 对其已有的解码画面执行检测。YOLO worker 属于 UAV 应用侧计算工具；此调用不能自动证明使用了完整 NDNSF-DI 分布式规划路径。[U2]

28.2 多视角任务

MultiViewRecognitionJob 持有不可变精确视图引用、模型 profile、目标、mission/job/attempt 和时间边界。UavDetectorProvider 验证获取的 Data，并通过 IMultiViewRecognitionAlgorithm 接口执行。算法输出融合结果和 Provider-owned 标注引用；成功、视图不足、验证/关联失败、模型不支持、计算失败、发布失败、超时与取消分别编码。[U6]

28.3 真实模型与功能替身

C++ 有 DeterministicMultiViewAlgorithm 用于显式功能契约检查。真实 MVCNN-family ONNX 工具路径在 tools/mvcnn_onnx_worker.py：检查注册模型摘要、视图摘要和输入形状，显式要求 CPUExecutionProvider；images 与 viewMask 输入在图内联合处理，输出 logits 和 pooledFeatures。常规多视角 job 要求不同 UAV 的足够视图；单视角配对 baseline 不能冒充多视角融合。[U6]

28.4 不能扩大解释的结果

C++ 算法注入接口存在不证明真实 ONNX worker 已在每个默认 GUI/Provider 路径中接入；工具 fixture 的功能结果也不证明真实飞行准确率。跨进程获取、候选竞争、单一终端 owner、结果/标注发布和失败终态需在对应网络夹具中分别验证。

29 跨模块协作的完整路径

用同一套 Core 机制连接领域任务与命名数据

29.1 UAV 录像路径

GS 经 Core 认证调用 Drone 的视频控制；Drone 发布受保护媒体 Data，可选内嵌 Repo 保存原始 packet 与留存 manifest。GS 查询录像清单，按精确引用取回并验证/解密后播放。UAV 定义媒体与保留语义；Repo 管字节/目录；Core 管调用、身份与传输。[U2, U5, R1]

29.2 DI 模型意图路径

应用提交模型和输入；Python 当前编排通过 Core 收集认证 ACK，adapter/placement 决定图有效的角色分配；可信物化后经 Repo 发布制品，再提交 Core Selection。Provider 原生运行器获取依赖、执行并发布输出，terminal role 产生被调用者接收的结果。原生 requester 仍未接通同等全链路。[D2, D5, D7]

29.3 UAV 分布式分析路径

任务管理器记录 incident 和精确 evidence；通过 Core 协作收集分析 Provider，固定角色/terminal owner，Provider 拉取并验证证据后返回报告及标注引用。若某服务选择以 DI 实现计算，应通过明确的 DI 应用接口接入；不能因同仓库有 DI 就把全部 UAV 检测自动画成 DI 调用。[U4, U6]

共同约束	具体要求
控制/数据分离	服务请求传小型描述和引用；完整对象/媒体沿合适的数据面传输。
身份与摘要	调用者、发布者、Provider、对象、request、attempt、plan 等按各层契约绑定。
状态不混淆	存储已提交、模型已准备、资源已预留、飞行已执行、画面已显示是不同事实。
恢复有 owner	Core 清理调用；Repo 恢复已验证字节；DI 管执行/会话；UAV 管任务补偿。

30 构建、配置与部署边界

源码、宿主构建、容器候选和运行证据分别建立身份

部署形态	组件与前提
本地 Core 应用	C++ runtime、ndn-cxx、NFD、ndn-svs、NAC-ABE、信任模式与私有身份环境。
Repo-node	Core 服务、SQLite/存储目录、热点缓存预算；大制品 backend 按所选物化方式配置。
UAV 应用	Drone/GS 容器、GUI/媒体依赖、mock 或实际 MAVLink 连接、摄像头、可选 Repo。
DI Provider	Core、原生 DI 库、匹配模型 runner、ONNX Runtime/可选 CUDA、精确模型与制品引用。
MiniNDN	多进程 NFD/路由/身份隔离、拓扑及实验脚本；不能用单进程调用替代网络证据。
Tiger/SIF	封存源码、容器内原生构建、候选身份、Apptainer/Slurm、实际节点设备和执行记录。

30.1 工具链一致性

宿主原生构建按仓库规则使用 system compiler/binutils 和匹配的 Boost 头/库，先完成 NAC-ABE 依赖闭包，再构建 Core 与 Python 扩展。改变 C++ ABI 后，需要重新构建依赖对象并核对实际加载的库；导入成功和 SONAME 不变都不能独立证明 ABI 一致。[O1]

30.2 容器与集群

Tiger 的唯一维护入口为 Experiments/TigerCluster/。适配器、job/profile、images 和 results 按该目录契约组织；共享组件仍由 packaging 维护。镜像原生组件须在容器 builder 内编译，不使用宿主 Python.h、venv 或宿主.so 替代镜像依赖。开发机负责代码/静态检查/测试，实验机负责 SIF 构建和 Tiger 实验。[O2]

30.3 配置、身份和数据

私钥、模型、数据库、缓存、运行日志与镜像不因文档生成进入版本库。模型名、设备参数和服务路由需与实际被加载/注册的内容核对；本轮没有启动 NFD/MiniNDN、构建原生依赖或向集群提交任务。

31 实现状态与验证口径

把已有代码、接线状态和运行资格分开报告

范围	本版可支持的结论
Core	已有通用调用、Targeted、协作、流、安全及在线授权组件；本文核对主要生产路径与状态分支。
Repo	已有 C++ 对象存储/服务、Python 编排和制品 API；具体后端、目录操作及控制模式存在差异。
UAV	已有飞控/GUI/任务/视频/Repo/识别组件和研究夹具；真实硬件和模型接线按入口分别确认。
DI	Python 应用编排与 C++ Provider 执行组件并存；原生请求全链路仍未接通，不能声明迁移完成。
本轮文档	源码核对、PDF 构建与文本/版面检查；没有新协议测试或性能测量。

31.1 测试入口

Core 检查位于 tests/unit-tests/ 与 tests/integration-tests/，包含 controller-revocation、controller-version-refresh、invocation-stream、stream-facade、opaque-selection 等主题。Repo 有存储后端/制品传输 C++ 检查与 test_spec164_* Python 检查；UAV 有 mission/collaboration/multiview 检查和 tools 运行夹具；DI 有 di-native-* 及 Python/experiment 检查。[V1]

这些名字是继续验证的入口，不是本轮通过清单。测试文件存在、一个 fixture 运行成功、完整网络结果和指定候选的硬件资格验收是不同证据等级。

31.2 活跃实现

当前 active Spec 为 182-native-di-python-bindings。真实来源/Repo 准备、报价消费、完整原生请求、流式接线、生成/会话与 Python caller 迁移仍由其 tasks/contracts 控制。本文不改写这些任务的验收状态。

失效快照的处理 代码变化后，source-baseline 摘要可能与当前工作树不再一致。这表示设计快照需要重新核对，而不是证明新代码有缺陷；目标文档也不能凭自动复制变成实现事实。

32 源码索引：Core 与授权

按入口定位，再用快照摘要确认正在阅读的版本

下列路径以仓库根为起点；正文中的引用 ID 指向这里和下一节。符号用于定位职责；详细文件清单与 SHA-256 见 source-baseline.json。

编号	规范文件与关注点
C1	docs/architecture.md; docs/ndnsf-core-app-boundary.md, 总体边界；以源码修正旧叙述。
C2	ndn-service-framework/ServiceContainer.hpp、LocalServiceRegistry.hpp；同目录 ServiceController/ServiceProvider。
C3	Core 的 ServiceUser.hpp、ServiceUser.cpp、ServiceProvider.cpp、NDNSFMessages.hpp；调用与协作。
C4	Core 的 GenericSelectionTxnStore.hpp、GenericSelectionTxnStore.cpp；不透明选择事务。
C5	Core 的 ServiceUser/ServiceProvider、RequestConfidentiality.cpp、HybridMessageCrypto.cpp；大数据和加密。
C6	Core 的 Stream.hpp、Stream.cpp、StreamFacade.hpp、StreamFacade.cpp；连续流、预测、FEC。
C7	Core 的 InvocationStream.hpp、InvocationStream.cpp；调用事件流与终态。
C8	Core 的 ExecutionLease.hpp、NetworkTelemetry.hpp、TimelineTrace.hpp 及 Provider；资源和观测。

32.1 授权引用

S1: ControllerVersion.cpp 的 isValid/compare。S2: ServiceController.cpp 的 getPolicyStatus。S3: 同文件的 reissueAbePolicies、rotateAbeGenerationAndReissuePolicies、reconcilePendingAbeRotation、advanceAuthorizationEpoch、revoke。S4: 同文件的 grant。S5: User/Provider 的 applyPermissionResponse。S6: installControllerStatus。S7: invalidateControllerScopedCaches 与 Provider 的 refreshProviderPermissionsAfterAdvance。S8: Controller 的 onPolicyStatusInterest、User/Provider 的 fetchPolicyStatusFromController/onPolicyStatusData。S9: PolicyRefreshCoordinator.cpp。S10: refreshNacDkeyForControllerStatus。S11: scheduleControllerStatusRefresh 与周期配置。S12: restorePersistedRuntimeStatuses。[均位于 Core 目录]

33 源码索引: Repo 与 DI

文件清单包含规范入口；不以临时 `compare/build` 副本替代当前源文件

编号	路径（每组根目录见下文）及关注点
R1	Repo README_ch.md、include/ndnsf-distributed-repo/RepoCore.hpp；对象与边界。
R2	Repo src/RepoNode.cpp、src/RepoClient.cpp；服务与控制入口。
R3	Repo src/RepoTypes.cpp；普通 SQLite/热点缓存后端。
R4	Repo include 中 RepoStoreBackend.hpp、FilesystemArtifactStore.hpp、ArtifactTransfer.hpp、ArtifactManifest.hpp。
R5	Repo pythonWrapper/py_repoclient/artifact_api.py；公开会话 API。
R6	同 Python 包 network_artifact_backend.py；真实网络发布、协作和 receipt 绑定。
R7	同 Python 包 orchestration.py、persistence.py 与 Repo src/RepoCore.cpp；目录、副本与后端。
D1	DI Python api/__init__.py、app_sdk/client.py、application.py、provider.py。
D2	DI Python app_sdk/placement.py、sdk/placement.py；ACK 驱动规划与 V3。
D3	DI C++ NativeRequestPreparation.cpp、NativeOfferAdmission.cpp、NativeV3Placement.cpp。
D4	DI C++ NativePlanSealer、NativeGrantClient/Verifier、ProtectedRuntime、NativeProtectedArtifactStore；封装和保护。
D5	DI C++ NativeProviderSession/Runtime/Handler、ProviderRoleWorker、TensorBundleCodec、DependencyWaitScheduler、CollectiveRuntime。
D6	DI C++ NativeEpochCoordinator、NativeConversationCoordinator、NativeStandaloneTokenizer；生成与会话。
D7	DI C++ NativeInferenceClient.cpp: dispatchOperation；active Spec182 tasks 与 evidence。
D8	DI C++ NativeInferenceProvider；Python app_sdk 与 sdk 的部署/扩展接口。
Repo 根为 NDNSF-DistributedRepo/；DI Python 根为 NDNSF-DistributedInference/ndnsf_distributed_inference/，DI C++ 根为 NDNSF-DistributedInference/cpp/ndnsf-di/。	

34 源码索引：UAV 与交付

应用服务、硬件/模型工具与运行入口分别定位

编号	规范路径与关注点
U1	UAV README_ch.md；shared/UavNames.hpp。
U2	UAV drone/DroneServiceContainer.inc.hpp、ground-station/GroundStationServiceContainer.inc.hpp 及各 App/Window。
U3	UAV shared/UavProtocol.hpp；状态与线协议类型。
U4	UAV shared/UavMissionSession.cpp、ground-station/UavIncidentCoordinator.cpp、drone/UavCollaborationParticipant.cpp。
U5	UAV shared/UavVideoPipeline.hpp/.cpp 及 Drone/GS 容器；媒体与留存。
U6	UAV shared/UavMultiViewRecognition.cpp、UavDetectorProvider.cpp、tools/mvcnn_onnx_worker.py。
O1/O2	AGENTS.md、docs/native-build-parallelism.md、Experiments/TigerCluster/README.md 与 docs/sif-build.md。
V1	根目录 tests/、Experiments/，各模块 examples/tools 及 active Spec evidence。

UAV 根为 NDNSF-UAV-APP/。O2 中的 docs/sif-build.md 相对 Experiments/TigerCluster/；其他交付路径相对仓库根。

34.1 模块维护文件清单

module-inventory.json 按 Core、UAV、DI、Repo 以及公共绑定/支持目录记录受版本库维护的文件路径、类型、大小及源码摘要；模型、原始日志和构建目录不因文档任务被复制进 Design。重点核对的规范文件另列于 source-baseline.json。

34.2 源码检索口径

CodeGraph 用于定位规范符号和调用者；出现 compare-old/compare-new、staging 或 build 副本时，以仓库维护的原始路径为准。章节中的运行限制来自具体实现分支，不是根据类名或 README 的目标描述推测。

35 文档维护与后续目标变更

当前设计随已核对代码更新；目标设计随用户明确决定更新

1. 当前正文保存在 `current-content.tex`，目标正文保存在 `target-content.tex`。首版完全相同；以后两份独立维护，不用共享正文把目标修改自动传播到当前设计。
2. 修改目标时，记录变更动机、涉及模块、与当前行为的差异、依赖和验证条件。未决定的方案保持待讨论，不把想法当作已接受契约。
3. 实现落地后，重新核对生产路径、状态/安全分支和实际证据，更新当前正文与源码快照；不能仅因为目标 PDF 已更新就宣称功能完成。
4. 运行 `python3Design/build.py` 重建两份 PDF，检查文本、分页、字体、表格和图。文档身份标题可以不同；R0 的技术正文必须一致。
5. 更新 `CHANGELOG.md`、`validation.md` 和相关 Spec 进度。本目录完整纳入 Git；每个 Spec 的设计影响记入 `spec-design-changes.md`，与正文和 PDF 同步提交。

35.1 覆盖清单与源码快照

`coverage-matrix.md` 记录四模块各功能域对应章节与主要入口；`module-inventory.json` 记录本次维护目录的文件清单，用于以后发现遗漏或新增组件。文件被列入清单不表示每一行都完成审计；正文区分主要流程源码核对与配置/工具入口登记。

`source-baseline.json` 记录重点核对文件的摘要、采样时提交和工作树差异。本地源码快照与 PDF 构建日志位于唯一的 `.codex-tmp/` 目录，供核对文档所描述的时点；它们不授予任何运行资格。

35.2 本文未引入的行为变化

本版不新增 SVS epoch 公告、不改变 grant 的本地失效范围、不补接原生 DI 请求链，也不扩大 Repo/UAV 现有接口能力。下一步由用户指定目标设计修改，明确新旧差异后再讨论实现与验收。

交付标准 两份完整中文 PDF、可编辑源文件、覆盖/源码记录和验证记录一起维护。文档完成与代码实现完成是两个独立状态。

36 API 阅读方法与覆盖边界

AC-01 · Core 接口契约

36.1 如何阅读

先看前 35 章的组件职责，再查本部分的入口契约，最后用 `api/` 下的完整声明参考核对重载、默认参数、类型和字段。API 清单是声明证据，不是运行资格。每个条目的 ID 绑定文件、符号和签名；签名改变会产生新 ID，应在 Spec 设计变更记录中关联旧、新条目。

36.2 范围

C++ 覆盖规范模块头文件的 `public/protected` 声明和数据字段；Python 覆盖规范包中公共命名的定义、带类型字段与可静态读取的导出。UAV 的容器内部接口明确标为 `application-internal`。宏展开、继承展开、运行时动态导出和 C++ ABI 不由此清单证明。测试 helper 不作为公开产品调用建议。

36.3 契约约定

签名中的 `const`、引用、`shared_ptr/unique_ptr`、`optional`、`std::function` 和默认值原样保留。Ms/ms 是毫秒，Ns 是纳秒；未带单位的计数不能按时间使用。空 `optional`、`false`、异常、错误回调和终端事件是不同结果，按接口说明处理。未声明的线程安全或重试保证不推断存在。

36.4 参考来源

组织方式参考 NFD Developer's Guide 的组件、数据结构、处理管线以及 Strategy API 的触发/动作说明。本次参考的是官方维护 PDF（Revision 12 标为 TBD）；只借鉴组织方法，NDNSF 契约全部依据本项目源码。详见 `references.md`。

完整重载、数据字段和原始注释：`api/core-reference.md`。

37 Core API：容器与角色生命周期

AC-02 · Core 接口契约

API-ee68261266a4 public

```
template <typename RequestT, typename ResponseT, typename HandlerT>
void
    addLocalService(const ndn::Name& serviceName, HandlerT&& handler)
```

源码：`ndn-service-framework/ServiceContainer.hpp`，第 73 行。

API-04ad67003514 public

```
void
    start()
```

源码：ndn-service-framework/ServiceContainer.hpp, 第 331 行。

API-cd596b18ee08 public

```
void
  stop()
```

源码：ndn-service-framework/ServiceContainer.hpp, 第 365 行。

37.1 入口与参数

RuntimeConfig 中 containerIdentity、groupPrefix、controllerPrefix 与 trustSchemaPath 描述进程组成。addController/addUser/addProvider 以角色名登记共享或独占对象；use 系列借用外部对象，调用者必须保证其生命周期。addLocalService 的 RequestT/ResponseT 是进程内类型契约，不能跨网络序列化后仍假定同一 C++ 类型身份。

37.2 处理流程

先构造 Face/身份环境，再登记角色和 helper hooks，最后 start。容器调用各角色初始化并启动生命周期 hook；stop 负责相应停机。运行服务请求通过 User/Provider，可信同进程调用通过 LocalServiceRegistry，不经网络权限握手。

37.3 状态与失败

启动后再改变受保护的注册集合会被 ensureNotStarted 拒绝；空角色、空对象和缺失角色分别按具体方法抛错。此类异常不是远端超时。对象容器不承诺所有外部业务线程已被取消；应用 hook 应承担自己的 worker 排空与资源回收。

37.4 扩展点

新增模块通过本地服务或生命周期 hook 组合，避免让容器理解模型图、航点或 Repo manifest。先核对 start/stop 的异常与重复调用分支，再设计自定义 hook 的幂等性。

完整重载、数据字段和原始注释：api/core-reference.md。

38 Core API：请求发起与 Targeted 调用

AC-03 · Core 接口契约

API-1009e77682c2 public

```
ndn::Name RequestServiceTargeted(const ndn::Name& provider,
                                  const ndn::Name& serviceName,
                                  ndn_service_framework::RequestMessage requestMessage,
                                  int timeoutMs,
                                  TimeoutHandler onTimeout,
                                  ResponseHandler onResponseHandler);
```

源码：ndn-service-framework/ServiceUser.hpp，第 718 行。

API-efa9e29aae17 public

```
template<typename RequestT, typename ResponseT>
ndn::Name RequestServiceTargeted(const ndn::Name& provider,
                                const ndn::Name& serviceName,
                                const RequestT& request,
                                std::function<void(const ResponseT&)> onResponse,
                                std::function<void()> onTimeout,
                                int timeoutMs)
```

源码：ndn-service-framework/ServiceUser.hpp，第 977 行。

38.1 参数与返回

provider 与 serviceName 均为 NDN Name；requestMessage 或模板 RequestT 携带应用载荷，timeoutMs 为调用期限。返回 ndn::Name 是请求标识，不是结果。低层重载使用 TimeoutHandler/ResponseHandler；类型化重载传递解码后的 ResponseT，不应互换回调签名。完整普通 RequestService 重载见 Core 声明参考。

38.2 调用流程

普通调用发布 Request，收集已认证 ACK，执行选择后发送 Selection，再接受绑定该请求的 Response。Targeted 利用已有 Provider/token 关联；没有可用关联时的 bootstrap/重建行为由 User 实现，指定 provider 并不跳过授权。

38.3 完成与错误

调用方保持回调捕获对象有效，直到完成或超时。NDN Nack、取数失败、权限拒绝、业务错误和 deadline 到期需要按各自通路观察；不能把拿到 requestId 当作服务已执行。getRequestStatus 和生命周期回调用于本地状态观测，不构成远端执行证据。

38.4 线程与安全

User 提供 postToIo/isOnIoThread；涉及 Face 的操作遵守其执行上下文。模板请求/结果须满足具体序列化路径。request、service、Provider 证书及 ControllerVersion 的绑定由 Core 检查，应用不能只用 payload 自报身份替代。

完整重载、数据字段和原始注释：api/core-reference.md。

39 Core API: Provider 注册与作用域回收

AC-04 · Core 接口契约

API-c3a3fd9da1b0 public

```
ServiceRegistration addScopedService(const ndn::Name& serviceName,
                                     AckStrategyHandler ackHandler,
                                     RequestHandler requestHandler,
                                     ServiceInvocationMode invocationMode);
```

源码：ndn-service-framework/ServiceProvider.hpp, 第 856 行。

API-23e6ea05d4d6 public

```
ServiceRegistration addScopedService(const ndn::Name& serviceName,
                                     AckStrategyHandler ackHandler,
                                     RequestHandler requestHandler,
                                     ServiceMode mode);
```

源码：ndn-service-framework/ServiceProvider.hpp, 第 860 行。

API-33314e8f5bbb public

```
ServiceRegistration addScopedCollaborationHandler(
    const ndn::Name& serviceName,
    std::vector<CollaborationRole> allowedRoles,
    AckStrategyHandler ackHandler,
    CollaborationHandler handler);
```

源码：ndn-service-framework/ServiceProvider.hpp, 第 869 行。

39.1 参数与返回

serviceName 是受注册所有权约束的名称；ackHandler 决定请求阶段的资格/报价，requestHandler 或 CollaborationHandler 执行业务。ServiceMode 与 ServiceInvocationMode 分别表达服务与调用模式。返回 ServiceRegistration 是 move-only RAI handle，应用须保存它以维持该注册代次。

39.2 触发顺序

认证 Request 到达后执行 ACK 路径；被选定且 Selection 绑定校验通过后才进入执行 handler。协作 handler 获得角色和 assignment；同一个名字的 legacy/scoped 注册不能覆盖仍存活的 scoped owner。

39.3 关闭语义

close 或 handle 析构先关闭 generation，阻止新的 ACK/Selection/排队执行使用该代次，再将清理投递到 Face。Provider 销毁后关闭旧 handle 为无害操作。关闭注册不等于历史 Data 从网络缓存消失。

39.4 线程与扩展

这些注册方法按头文件约定只在 Face 线程或事件循环启动前调用。handler 可以使用线程池，但必须遵守 generation、deadline 和停止边界。新增模型/飞控业务通过 handler 扩展，不修改 Core 的领域语义。

完整重载、数据字段和原始注释：api/core-reference.md。

40 Core API：延迟规划与协作提交

AC-05 · Core 接口契约

API-c9045b5b03e6 public

```
ndn::Name BeginCollaboration(const ServiceName& service,
                             const RequestPayload& initialRequest,
                             int ackCollectionTimeMs,
                             int timeoutMs,
                             CollaborationAckClosedHandler onAckClosed,
                             ResponseHandler onFinalResponse,
                             TimeoutHandler onTimeout,
                             const RequestId& requestId = RequestId());
```

源码：ndn-service-framework/ServiceUser.hpp，第 858 行。

API-3ab0d3d87607 public

```
bool CommitCollaborationPlan(const RequestId& requestId,
                             const std::string& ackClosedDigest,
                             CollaborationPlan plan);
```

源码：ndn-service-framework/ServiceUser.hpp，第 887 行。

40.1 参数与状态

BeginCollaboration 的 ackCollectionTimeMs 控制 ACK 窗口，timeoutMs 控制整个请求期限；requestId 可显式提供。onAckClosed 收到不可变 ACK_CLOSED，包括可用于提交的摘要。返回请求名后仍处于异步流程，不能立即假定已经选择 Provider。

40.2 提交契约

应用只在相应 ACK 关闭后构造 CollaborationPlan，并使用同一 requestId 与 ackClosedDigest 调用 CommitCollaborationPlan。bool 表示提交接口的接受结果，不表示所有 Provider 已完成。计划内角色、assignment 和最终结果接受规则由应用定义并由 Core 传送和认证。

40.3 错误与副作用

过期、错误请求或错误闭包摘要不能作为可提交计划；业务计划失败应沿超时/失败路径收尾，不能复用旧闭包用于新 attempt。GenericSelectionTxnStore 的 prepare/commit 与应用协作计划是相关但不同层的事务。

40.4 时序示意

User Begin → Provider ACK → User ACK_CLOSED → 应用验证报价/规划 → CommitCollaborationPlan → Provider Selection 校验 → 各角色执行 → 终端结果。DI 在 ACK 后 inspect/规划；UAV 使用有限 job，不能把长期 mission 直接映射为永不结束的 invocation。

完整重载、数据字段和原始注释：api/core-reference.md。

41 Core API：调用流的事件和终态

AC-06 · Core 接口契约

API-9e9c476a768e public

```
template<typename RequestT, typename EventT, typename ResponseT>
std::shared_ptr<StreamedInvocationHandle<EventT, ResponseT>>
    RequestServiceStreaming(
        const ndn::Name& serviceName,
        const RequestT& request,
        StreamedInvocationOptions options,
        std::function<void(const EventT&)> onEvent,
        std::function<void(const ResponseT&)> onComplete,
        std::function<void(const StreamedInvocationError&)> onError,
        size_t strategy = ndn_service_framework::tlv::FirstResponding)
```

源码：ndn-service-framework/ServiceUser.hpp, 第 779 行。

API-2eccc93af394 public

```
bool publish(const EventT& event)
```

源码：ndn-service-framework/InvocationStream.hpp, 第 348 行。

API-2bdd4eb83edb public

```
void finish(const ResponseT& response, StreamFinishReason reason)
```

源码：ndn-service-framework/InvocationStream.hpp, 第 359 行。

41.1 返回与回调

RequestServiceStreaming 返回共享调用 handle；onEvent 接收 EventT，onComplete 接收最终 ResponseT，onError 接收 StreamedInvocationError。StreamedInvocationOptions 约束 deadline、队列和重试。模板类型须满足相应序列化要求。

41.2 Provider 动作

addStreamingHandler 提供 writer；publish 发送事件，finish 结束并提供最终结果，fail 产生错误终态。isCancelled、remainingDeadline 和状态值用于判断是否仍可发布。发布成功、事件送达、回调

消费和最终完成是不同观察点。

41.3 状态与背压

Core 管理 cursor、重排序、转录摘要、AEAD 与签名 Data，并通过有界队列限制等待事件。处理 terminal 后不能继续当作新流；取消、授权失效和超时均有独立关闭路径。调用流不等于独立连续视频流。

41.4 扩展约束

应用编码事件语义，Core 保留请求/attempt/ControllerVersion 绑定。业务若需要恢复，应明确恢复身份与新 attempt，不使用旧 handle 伪装连续成功。

完整重载、数据字段和原始注释：api/core-reference.md。

42 Core API：命名对象与连续流

AC-07 · Core 接口契约

API-160499aab055 public

```
ndn::Name publishSignedAppData(
    const ndn::Name& dataName,
    const ndn::Buffer& payload,
    ndn::time::milliseconds freshness = ndn::DEFAULT_FRESHNESS_PERIOD);
```

源码：ndn-service-framework/ServiceUser.hpp，第 637 行。

API-ef2281d2fe2d public

```
void fetchSignedAppData(
    const ndn::Name& dataName,
    const ndn::Name& expectedSigner,
    int timeoutMs,
    SignedAppDataHandler onData,
    SignedAppDataFailureHandler onFailure);
```

源码：ndn-service-framework/ServiceUser.hpp，第 660 行。

API-68706878d1a8 public

```
void push(std::shared_ptr<ndn::Data> signedData);
```

源码：ndn-service-framework/StreamFacade.hpp，第 46 行。

42.1 命名对象

publishSignedAppData 以 dataName 发布带签名载荷，freshness 使用 ndn::time::milliseconds；fetchSignedAppData 同时指定精确 dataName、expectedSigner 与 timeoutMs。成功回调和失败回调分

离，签名者校验不是只比名称字符串。

42.2 大对象与协作

`publishEncryptedLargeData`/`fetchEncryptedLargeData` 承担完整对象传输；`CollaborationContext` 的 `publishLargeNamed`、`fetchSignedExactData` 等在协作身份范围内使用。应用决定 `manifest` 和对象引用，`Core` 处理字节、签名、分段与授权。

42.3 连续流

`createStream`/`subscribeStream` 或 `live stream facade` 以 `stream identity`、`session` 与 `cursor` 定位数据。`StreamPublisher` `push` 接收带元数据的块，`stop/status` 管理运行；`PredictiveStreamSubscriber` 负责预测取数和统计，`Core` 已有通用不透明字节 FEC。

42.4 生命周期

发布者队列与订阅缓存有界；内容编码、可丢帧/必须完整等应用约束由 UAV 或其他模块选择。取到一个分段不等于完整对象，取到某个最新样本不意味着消费了所有历史事件。

完整重载、数据字段和原始注释：[api/core-reference.md](#)。

43 Core API：在线授权与版本状态

AC-08 · Core 接口契约

API-d5f6b3c470a6 public

```
bool grant(const ndn::Name& identity,
           const ndn::Name& serviceName,
           const ndn::Name& authorizationAttribute);
```

源码：`ndn-service-framework/ServiceController.hpp`，第 103 行。

API-3ad7bdf59cc1 public

```
bool revoke(const RevocationTarget& target);
```

源码：`ndn-service-framework/ServiceController.hpp`，第 95 行。

API-a6e1ee327414 public

```
PolicyStatusData getPolicyStatus(const ndn::Name& serviceName) const;
```

源码：`ndn-service-framework/ServiceController.hpp`，第 91 行。

43.1 参数

grant 接收身份、服务与精确授权属性三个 Name。revoke 接收有类型的 RevocationTarget，以区分身份、服务或证书范围。getPolicyStatus(serviceName) 返回该服务的状态载荷；真正发布时由 Controller 为外层 Data 签名。

43.2 返回与变更

bool 返回值用于区分是否产生变更或是否存在待完成处理；不能用它推断所有远端已安装新状态。grant-only 更新目标身份的完整策略并推进 ControllerVersion，保持 ABE generation；撤权推进版本并旋转公参、重发剩余有效策略，重复完成的撤权是 no-op。

43.3 失败与恢复

旋转失败保留撤权事实和 pending 处理，后续 grant/revoke 先协调恢复。PolicyStatusData 的全局版本与服务作用域分别解释；policyDigest 不是所有运行时授权变化的完整摘要。

43.4 传播与调用影响

远端通过 PolicyStatus 拉取、启用的周期发现或经过认证的业务 hint 获知版本，hint 不是权威。无关身份可跳过 DKEY 重取，但安装新服务版本仍可使请求/流缓存失效。具体分支见第 9–13 章，不能把局部取钥优化写成完全无感更新。

完整重载、数据字段和原始注释：api/core-reference.md。

44 Core API：执行租约、观测与并发

AC-09 · Core 接口契约

API-f52dcb87cd76 public

ExecutionLeaseResult

```
release(const std::string& leaseId, const std::string& providerEpoch,
        const std::string& requesterName, const std::string& idempotencyKey,
        uint64_t nowMs);
```

源码：ndn-service-framework/ExecutionLease.hpp，第 145 行。

44.1 数据边界

ExecutionLease 及 ProviderAdmissionLeaseTable 管理有界 admission/execution 资源；generic admission token 不是服务权限，也不是 DI execution grant。请求的 lease 必须绑定具体身份、请求、服务与有效期，不能仅把 ACK 的 capacity 视作永久预留。

44.2 观测入口

User/Provider 的生命周期 callback 与 `getActiveRequestStatuses`、`getProviderAdmissionCounters`、`consumeRuntimeDiagnostics` 描述本地运行。TimelineTrace/NetworkTelemetry 支持跟踪，不改变协议接受结果。Ms 与队列大小分别为时间和数量。

44.3 线程

`setHandlerThreads`、`setAckProcessingThreads`、`setAckThreads` 的作用域不同；设置更多线程不使全部 Face API 变为任意线程安全。异步关闭要先禁止新任务，再停止/排空 worker 和回调捕获。

44.4 检查

使用 `scoped registration` 与租约生命周期检查确认关闭、重复释放和过期行为；统计值不能替代 selected Provider 的实际执行/终端结果。完整类型和 setter 参数见 Core 参考。

完整重载、数据字段和原始注释：api/core-reference.md。

45 Repo API: 对象与精确 Data packet

AC-10 · Repo 接口契约

API-3690bad70c85 public

```
static RepoObjectManifest put(RepoNode& node,
                             const std::string& objectName,
                             const std::vector<uint8_t>& payload,
                             StoreOptions options = {});
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoClient.hpp, 第 33 行。

API-208878700957 public

```
static std::vector<std::vector<uint8_t>> getDataPackets(
    const RepoNode& node,
    const RepoObjectManifest& manifest);
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoClient.hpp, 第 62 行。

45.1 参数与返回

put 的 objectName 是对象身份，payload 是字节，StoreOptions 指定 objectType、replicationFactor、replicaNodes 和 policyEpoch；返回 RepoObjectManifest。静态本地、LocalServiceRegistry 与网络 helper 的重载不共享同一种传输/异常时序。

45.2 packet 契约

putDataPacket 保存签名 Data 的原始 wire, 并使用其中编码的精确名称。getDataPackets 按 manifest 的有序完整名称读取; 索引不完整、名称不一致等情况在返回整组结果前拒绝。Repo 不重新解释或重签应用 payload。

45.3 存储与删除

普通对象默认 SQLite 加有界缓存; 插入先完成权威存储再更新缓存。remove 返回是否删除, 删除本地对象不意味着清除了所有副本或网络缓存。RepoCore 保持网络中立, RepoNode 拥有服务注册。

45.4 安全与单位

SHA-256、segmentCount、size、objectType 和 policyEpoch 不能互相替代身份字段; metadata 的声明不证明 payload 已经加密。调用者需要明确精确字节、签名者和授权要求。

完整重载、数据字段和原始注释: api/repo-reference.md。

46 Repo API: 制品上传与提交

AC-11 · Repo 接口契约

API-c7c484fc1ed7 public-by-name

```
def publish_file(
    self,
    path: Union[str, Path],
    *,
    name: str,
    expected_sha256: str,
    replicas: int = 1,
    verification: str = "signed-manifest",
    resume: bool = True,
    on_progress: Optional[ProgressObserver] = None,
    idempotency_key: str = "",
    policy_epoch: str = "default",
    timeout_ms: Optional[int] = None,
    control: ArtifactControlOptions = ArtifactControlOptions(),
    cancellation: Optional[ArtifactCancellationToken] = None,
) -> ArtifactPublishResult:
```

源码: NDNSF-DistributedRepo/pythonWrapper/py_repoclient/artifact_api.py, 第 844 行。

API-91dcd6af85dd public-by-name

```
def begin_upload(
    self,
    descriptor: ArtifactDescriptor,
```

```

*,
on_progress: Optional[ProgressObserver] = None,
cancellation: Optional[ArtifactCancellationToken] = None,
) -> ArtifactUploadSession:

```

源码：NDNSF-DistributedRepo/pythonWrapper/py_repoclient/artifact_api.py, 第 819 行。

46.1 输入与输出

publish_file 要求 path、逻辑 name、expected_sha256; replicas 默认 1, verification 默认 signed-manifest, resume 默认 True。返回 ArtifactPublishResult; async 变体与高级 session 的状态/回调形式在 Repo 参考中分别列出。timeout_ms、取消 token 和 progress observer 是不同控制面。

46.2 显式会话

begin_upload(descriptor) 返回 ArtifactUploadSession; 随后 upload_file, 再 commit。transfer/upload 完成并不等于对外可见, commit 校验 receipt、选定副本集合、operation 和 digest 后才产生成功引用。abort(preserve_progress=True) 可以保留可恢复 staging。

46.3 网络限制

公开 control 允许表达多种模式, 但当前 network backend 的整制品发布拒绝 TARGETED 并要求 Collaboration。不要因签名接受一个选项便推断所有 backend 都实现该模式。

46.4 恢复身份

幂等键、artifact/root digest、chunk 参数和 lease 联合约束恢复; 同路径文件被替换后不能沿用旧进度。进度回调反映阶段, 不替代完整 hash 与副本 receipt 校验。

完整重载、数据字段和原始注释: api/repo-reference.md。

47 Repo API: 下载、验证与传输状态机

AC-12 · Repo 接口契约

API-1707faa2f3ea public-by-name

```

def fetch_file(
    self,
    reference: ArtifactReference,
    destination: Union[str, Path],
    **kwargs,
) -> ArtifactFetchResult:

```

源码：NDNSF-DistributedRepo/pythonWrapper/py_repoclient/artifact_api.py, 第 960 行。

API-6f363704fd88 public

```
ArtifactSegmentDisposition
```

```
    receive(uint64_t segmentNo, uint64_t logicalBytes, uint64_t wireBytes,  
            uint64_t nowMs);
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/ArtifactTransfer.hpp, 第 78 行。

API-b8a723742c4e public

```
void markVerified(uint64_t segmentNo);
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/ArtifactTransfer.hpp, 第 82 行。

47.1 获取结果

fetch_file 接收不可变 ArtifactReference 与 destination; kwargs 由内部 begin_fetch/会话参数继续约束, 不能把 **kwargs 视为任意选项均可接受。begin_fetch 返回下载 session; transfer 后 commit 才物化最终可见文件。

47.2 接收与验证

AdaptiveArtifactTransfer 将 poll 的调度、receive 的接收与 markVerified 的校验分开。收到字节先进入 verification backlog; 摘要或签名失败走 reject/fail。窗口背压限制未完成与待验证数据, 并处理 expire 超时。

47.3 恢复状态

ArtifactResumeSession 的 restoreVerified、markVerified、renewLease、resume、cancel、expire、complete/fail 维护同一身份下的状态。未验证范围不能持久化为已完成; 重复、超期和跨身份恢复必须拒绝。

47.4 存储边界

PayloadStore/MetadataStore 与 FilesystemArtifactStore 维护 staging 和元数据, 区别于普通对象 SQLite。重试需保存 first failure; 旧缓存存在不证明恢复后的文件与引用匹配。

完整重载、数据字段和原始注释: api/repo-reference.md。

48 Repo API: 目录、副本与缓存控制

AC-13 · Repo 接口契约

API-b28b415568be public

```
RepoCatalogDelta catalogSnapshot() const;
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp，第 60 行。

API-e296daab62b3 public

```
RepoCatalogDelta catalogDelta(uint64_t sinceEpoch) const;
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp，第 62 行。

API-2c18852cdee0 public

```
RepoCatalogEntry catalogLookup(const std::string& objectName) const;
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp，第 64 行。

48.1 目录数据

catalogStatus 报告本 Repo 的模式、epoch 和对象数；snapshot 是完整本地目录，delta 按目录 epoch 提供变化，lookup 以名称/类型/类别等字段过滤。目录 epoch 不是 Core ControllerVersion。

48.2 跨 Repo 行为

目录交换、同步和 repair 的 Python 编排不因为 C++ DTO 存在就自动启动。Replica metadata 声明、实际 Data 可读性、保留/删除状态必须分别核对。共享目录不意味着每个节点都拥有全部对象。

48.3 删除与保留

应用指定 object class、retention、repairAllowed 等语义，Repo 执行保存和删除规则；tombstone 不能被旧 repair 错误复活。catalog/control 入口的 C++ 与 Python 支持集合分别在参考中查证。

48.4 失败

缺失对象、校验错误、过期 lease、副本不足和控制命令拒绝不是同一错误。调用者按明确 operation/status 响应处理，不以空结果一概解释全部失败。

完整重载、数据字段和原始注释：api/repo-reference.md。

49 DI API: Python 应用入口与原生句柄

AC-14 · DI 接口契约

API-0ad867060005 public-by-name

```
def request(
    self,
    *,
    model,
    task,
```

```

        input,
        timeout_ms: int,
        options=None,
        objective=None,
        constraints=None,
        request_id: str = "",
        strategy=None,
    ):

```

源码：NDNSF-DistributedInference/ndnsf_distributed_inference/app_sdk/client.py, 第 307 行。

API-375b92c9a402 public

```

NativeInferenceHandle request(
    const NativeModelRef& model,
    const NativeApplicationInput& input,
    std::shared_ptr<const NativeModelSplitStrategy> splitStrategy,
    std::shared_ptr<const NativePlacementStrategy> placementStrategy,
    const NativeRequestOptions& options);

```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp, 第 121 行。

API-c0c25988d534 public

```

NativeInferenceResult result(std::chrono::milliseconds waitTimeout) const;

```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp, 第 95 行。

49.1 应用入口

APPClient.request 以 model、task、input、timeout_ms 为必需业务输入，options/objective/constraints/strategy 按路径处理。InferenceClient 的 request_model、request_task 和 request(deployment, ...) 是不同入口；api 包的重导出不改变真实 owner。

49.2 原生数据

NativeApplicationInput 区分 Inline 与 RepositoryReference，携带任务和 schema 摘要；NativeRequestOptions 的 timeoutMs 默认 30000、ackTimeoutMs 默认 5000。NativeInferenceHandle 返回请求标识、Pending/Succeeded/Failed/Cancelled 状态；result(waitTimeout) 的等待期限不等于修改底层总 deadline。

49.3 当前限制

原生生产 dispatch 仍返回 NATIVE_REQUEST_PIPELINE_NOT_READY；已有 executor、timer、句柄与 Provider runtime 不能证明完整 C++ requester 链已接通。NativeDiError 保留 code/domain/boundary/request Id/attempt，调用者不应只比较异常文案。

49.4 生命周期

cancel 请求取消, observe 注册事件观察, close 停止 client; 捕获资源须覆盖异步生命周期。Python 编排和 native 迁移的实际差异由 Spec182 跟踪, 目标 PDF 本轮仍不自动增加尚未获用户单独确认的迁移设计。

完整重载、数据字段和原始注释: [api/di-reference.md](#)。

50 DI API: 模型适配与角色放置扩展

AC-15 · DI 接口契约

API-da75fc521e8b public

```
virtual std::vector<NativeSplitCandidate> enumerate(
    const NativeModelDescriptor& model,
    const NativeGraphSnapshot& graph,
    const NativeCandidateBudget& budget) const = 0;
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanning.hpp, 第 172 行。

API-0f4ac0e590fd public

```
virtual NativeRolePlacementProposalV3 proposeRoles(
    const NativeOfferBindingContext& context, const std::string& ackClosedDigest,
    const std::vector<NativeSelectionRoleV3>& roles,
    const std::vector<NativeAdmittedOfferV3>& offers, std::uint64_t nowMs) const = 0;
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanning.hpp, 第 183 行。

50.1 切分输入

enumerate 输入模型描述、已检查的图快照和候选预算, 输出 NativeSplitCandidate 集合。候选描述应与 adapter 的真实图、工件及预算绑定; 不是先任意创造角色再让执行器猜测如何运行。

50.2 放置输入

proposeRoles 消费 NativeOfferBindingContext、ackClosedDigest、角色、已 admission 的 ProviderOfferV3 和 nowMs, 返回带身份与验证信息的 V3 proposal。策略只能选择当前报价允许的角色/设备组合, 不能从 capability 字符串推断现成 runtime。

50.3 实现时序

ACK_CLOSED → adapter inspect/候选枚举 → offer admission → placement → artifacts/plan seal。planning graph 与 canonical ONNX graph 是可不同的身份空间; 引用和 recipe 应绑定到正确对象。

50.4 扩展规则

新增策略实现 identity 与虚接口，保持确定的输入绑定、预算和拒绝行为。已有 NativePreSplitFirstPlacement 不代表所有模型都支持同一种切分；ONNX、Qwen、YOLO、Python llama 的后端支持分别核对。

完整重载、数据字段和原始注释：api/di-reference.md。

51 DI API：准备、发布后再绑定与封存

AC-16 · DI 接口契约

API-2d8e39c7fc67 public

```
NativeArtifactBinding ensureArtifacts(const NativeInspectedModel& model,
                                     const NativeSplitCandidate& candidate,
                                     const NativeRolePlacementProposalV3& proposal,
                                     const NativeRequestControl& control) const;
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp，第 121 行。

API-d192a8a670da public

```
static std::vector<NativeSelectionRoleV3> bindPublishedRoles(
    const NativeInspectedModel& model, const NativeSplitCandidate& candidate,
    const std::vector<NativeSelectionRoleV3>& roles, const NativeArtifactBinding& artifacts);
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp，第 128 行。

API-1df2153e06a6 public

```
static NativePlacementPlanCore sealCore(
    const NativePlanningSnapshot& snapshot,
    const NativePlacementProposal& proposal,
    const NativePlanSealingInputs& inputs);
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanSealer.hpp，第 114 行。

51.1 准备职责

prepareInput 绑定 task/schema、payload 或 Repo reference 及 steady-clock deadline；inspectModel 获取真实模型/图身份；prepareRoles/validateRoles 校验候选角色。NativeRequestControl 将 deadline、request/attempt 和取消检查带入准备流程。

51.2 发布后的身份

ensureArtifacts 使用完整 candidate、proposal 和 control，不能只验证返回摘要格式。当前准备 owner 会核对发布后的业务 root 原始字节与模型/profile/source/package 等身份；业务 root digest 与 Core 传输 manifest 是不同对象。

51.3 再绑定

bindPublishedRoles 在保持角色、候选、Provider 和设备不变的条件下，将最终 manifest/recipe 绑定回角色。旧 exact-reuse 证明若不能覆盖新 recipe，应拒绝，不能沿用旧 ready 标签。详细字段见当前头文件和 API 参考。

51.4 封存与边界

sealCore、grantView、finalizeSecurity、project、encode 构成计划/安全投影的不同步骤。端口测试通过不等于真实 catalog/Repo、Core publisher 与 requester 默认链已接通；新准备/发布组件的接线状态仍需 Spec182 证据。

完整重载、数据字段和原始注释：api/di-reference.md。

52 DI API: Provider 注册、授权与执行

AC-17 · DI 接口契约

API-b5abfc9b9043 public

```
NativeServiceRegistration serve(const NativeServiceDefinition& service,
                                const NativeProviderHandlerConfig& config);
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceProvider.hpp，第 98 行。

API-859789c40049 public

```
void close() noexcept;
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceProvider.hpp，第 64 行。

API-070f116bfa58 public

```
NativeGrantVerificationResult
verifyAndUnwrapNativeGrant(const std::string& wireJson,
                            const std::string& authorityPublicKeyRaw,
                            const NativeRecipientKey& recipientKey,
                            const std::string& providerIdentity,
                            const std::string& requestId,
                            std::uint64_t attempt,
                            const std::string& planCoreDigest,
                            const std::string& modelManifestDigest,
                            const std::string& protectionEpoch,
                            std::uint64_t nowMs,
                            const std::string& expectedAuthority = "",
                            const std::string& expectedGrantDigest = "");
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeGrantVerifier.hpp，第 57 行。

52.1 服务入口

serve 接收 NativeServiceDefinition 与 NativeProviderHandlerConfig, 返回 NativeServiceRegistration; serviceName、模型支持和运行配置由 DI 声明, 再交给 Core scoped registration。注册 handle 和 generation 属于实际生命周期, 不只是名称句柄。

52.2 执行链

NativeProviderHandler → NativeProviderSession → NativeProviderRuntime/ProviderRoleWorker → model runner。角色使用 assignment/dependencyIo, 准备和执行保留 request/attempt/grant 绑定; dependency watcher 的就绪不等于所有集群节点都就绪。

52.3 安全

verifyAndUnwrapNativeGrant 验证认证上下文与投影, 再解包运行所需授权; NativeProtectedArtifactStore/ProtectedRuntime 分担受保护工件与运行记录。服务调用权限、execution lease 与 request-scoped grant 是三种不同授权/资源约束。

52.4 失败与关闭

错误 grant、过期 lease、角色不符、模型未准备和执行错误保留不同边界。关闭注册要阻止过期 generation 的 queued work 发布结果。默认模型配置可能使用 plaintext-v1, 不能仅因存在 ProtectedRuntime 就声称所有工件均已加密。

完整重载、数据字段和原始注释: `api/di-reference.md`。

53 DI API: 生成、KV 与会话延续

AC-18 · DI 接口契约

API-b5b8e5590c3c public

```
void cancel();
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp`, 第 96 行。

53.1 状态 owner

NativeEpochCoordinator、GenerationEpochLineage、DecodeStateIdentity 与 ConversationStateBinding 分别管理 generation 轮次、谱系、解码状态及会话绑定。QwenGenerationSession 处理模型执行/KV, tokenizer 和采样策略属于 DI, 不进入 Core。

53.2 身份与输出

KV cache 的身份包含模型、工件、adapter/runner、role、rank、split、tokenizer 及 lineage 等契约字段; 不能按文件路径或请求号单独复用。token 事件、文本片段、stream complete 与 inference result

的最终返回区分处理。

53.3 暂停和恢复

pin、release、restore 与会话 journal 是不同阶段；恢复需核对绑定和保存时状态，不能静默把旧 cache 接到新模型或新 recipe。生成取消应阻止后续 token/终态误发布。

53.4 验证边界

当前原生 generation/tokenizer 组件已有局部实现和定向证据，但完整 requester/no-Python、流式与会话资格由 Spec182 控制。本轮 API 文档不升级这些运行结论。完整方法与字段分别见相应 C++ 及 Python 声明参考。

完整重载、数据字段和原始注释：api/di-reference.md。

54 UAV API：飞控边界与操作权限

AC-19 · UAV 接口契约

API-b1e46a3def66 public

```
virtual Fields sendMavlink(const std::vector<uint8_t>& frame,
                          const std::string& commandName) = 0;
```

源码：NDNSF-UAV-APP/drone/DroneServiceContainer.inc.hpp，第 8 行。

API-c4f6835e7d67 public

```
Fields
    sendMavlink(const std::vector<uint8_t>& frame,
                const std::string& commandName) override
```

源码：NDNSF-UAV-APP/drone/DroneServiceContainer.inc.hpp，第 79 行。

API-76b58451421c public

```
Fields
    sendMavlink(const std::vector<uint8_t>& frame,
                const std::string& commandName) override
```

源码：NDNSF-UAV-APP/drone/DroneServiceContainer.inc.hpp，第 314 行。

54.1 接口性质

FlightControllerBackend 位于 DroneServiceContainer 的应用内部实现，sendMavlink 接收原始 MAVLink frame 和 commandName，返回 Fields。Mock 与 UDP/串口实现具有不同外部副作用；同一签名不表示同一硬件资格。

54.2 网络入口

Ground Station 构造命令，经 /UAV/MAVLink/Execute 等 NDNSF 服务发送，Drone 的 handler 校验身份、操作权限和 readiness 后转交 backend。Fields 是应用键值协议，不等于类型完整的安全证明。

54.3 三层前置条件

Core 的 User/Provider 权限、UAV OperatorAuthorityLease、飞行 readiness/safety gate 分别独立。操作 lease 的 issuer 不是 Core Controller epoch；GPS 应取飞控 telemetry，不能用任意业务参数冒充真实车辆状态。

54.4 生命周期与失败

持续接收链路、串口/UDP worker、Qt UI 回调有不同执行上下文。链路断开、设备拒绝、操作 lease 到期和服务超时应保留原边界。新增飞控 backend 需实现能力、状态和停机行为，不在 Core 中加入 MAVLink 指令解释。

完整重载、数据字段和原始注释：api/uav-reference.md。

55 UAV API：任务、补偿与终端报告

AC-20 · UAV 接口契约

API-9439cf6844b2 public

```
bool compensateMissingParts(std::string* reason = nullptr);
```

源码：NDNSF-UAV-APP/shared/UavMissionSession.hpp, 第 49 行。

API-8bc23c85a02b public

```
bool acceptTerminalReport(const UavTerminalReport& report,
                          std::string* reason = nullptr);
```

源码：NDNSF-UAV-APP/shared/UavMissionSession.hpp, 第 60 行。

API-f10389b9d21d public

```
bool acceptReport(const UavTerminalReport& report, uint64_t nowMs,
                  std::string* reason = nullptr);
```

源码：NDNSF-UAV-APP/ground-station/UavIncidentCoordinator.hpp, 第 62 行。

55.1 任务模型

UavMissionSession 是长期 mission owner，包含 part、stream、incident 和有限 job。assignPart/markPartExecuting/completePart/markPartMissing 维护任务分片；compensateMissingParts 只处理

missing 集合，新 invocation 获取新 token/assignment。

55.2 返回与原因

这些 bool 方法表示本地状态迁移/报告是否被接受；可选 reason 输出诊断，false 不能等价于车辆已安全停止。acceptTerminalReport 核对 mission/job/attempt、计划与 terminal owner，旧 attempt 的成功报告不能完成新任务。

55.3 恢复

snapshot/restore 和 reconcileVehicleAndStreams 将持久任务状态与实际飞控/流状态对照；恢复不是把保存的 Active 标记直接当作仍在执行。UavIncidentCoordinator 的 retryAnalysis 和 retriesUsed 管理有界分析重试。

55.4 协作扩展

新增任务类型保持 mission/part/job 各自身份和状态机；使用 Core Begin/Commit 协作原语，应用定义航点和证据。不要把长期 mission 绑为一个无限存活的 Core invocation。

完整重载、数据字段和原始注释：api/uav-reference.md。

56 UAV API：视频管线、遥测与录像

AC-21 · UAV 接口契约

API-a064d74ab888 public

```
bool push(UavVideoFrame frame);
```

源码：NDNSF-UAV-APP/shared/UavVideoPipeline.hpp, 第 65 行。

API-11ef609f2477 public

```
std::optional<UavVideoFrame> popLatest();
```

源码：NDNSF-UAV-APP/shared/UavVideoPipeline.hpp, 第 66 行。

API-730501b40b2c public

```
TelemetryAdmissionResult admit(const std::vector<uint8_t>& wire,
                                uint64_t receivedTimestampNs);
```

源码：NDNSF-UAV-APP/shared/UavSensorStreams.hpp, 第 57 行。

56.1 媒体参数

视频 pipeline 区分 capture/decode、access unit、PTS、keyframe 和 sessionGeneration；BoundedLatestFrameQueue 以最新帧策略控制积压，不能用于必须完整保留的制品下载。GStreamer 与

LegacyPipe capability 探测决定可用后端。

56.2 遥测数据

CompactTelemetrySample 的 latitudeE7/longitudeE7、altitudeMm、groundSpeedMmps、batteryPer-mille 明确单位；sampleId 与 sourceTimestampNs 用于去重/新旧判断。LatestTelemetryAdmission::admit 返回 valid/stateAdvanced/newSample/duplicate/outOfOrder/ageNs/reason，不以单一 bool 概括全部状态。

56.3 完整块

CompleteAcousticBlockAdmission 对不透明声学块判断完整性和截止期；数据来源可以包括 Core 的恢复 provenance。Core 处理签名/FEC，UAV 判断一帧、一份遥测或一个完整块的领域可用性。

56.4 录像链

录像保存原始带签名/保护的 packet 与应用 manifest，再通过 Repo API 写入。录制开关独立于直播显示；gap、retention 和 playback 游标属于应用管理。Camera/Repo 完整产品链不能自动外推所有传感器录制都已实现。

完整重载、数据字段和原始注释：api/uav-reference.md。

57 UAV API：检测与多视角识别扩展

AC-22 · UAV 接口契约

API-1da9ec7348c6 public

MultiViewExecutionResult

```
executeMultiViewJob(const MultiViewRecognitionJob& job,
                    const MultiViewModelProfile& profile,
                    const std::vector<VerifiedMultiViewInput>& views,
                    const ndn::Name& terminalOwner,
                    const IMultiViewRecognitionAlgorithm& algorithm =
                        DeterministicMultiViewAlgorithm{});
```

源码：NDNSF-UAV-APP/shared/UavMultiViewRecognition.hpp，第 195 行。

API-01be92408d26 public

```
bool acceptMultiViewResult(const FusedRecognitionResult& result, uint64_t nowMs,
                          std::string* reason = nullptr);
```

源码：NDNSF-UAV-APP/ground-station/UavIncidentCoordinator.hpp，第 66 行。

57.1 参数与输出

executeMultiViewJob 输入 job、model profile、已验证 views、terminalOwner 和算法对象，输出 MultiViewExecutionResult。默认算法是 DeterministicMultiViewAlgorithm 测试/研究夹具，不能宣称默认执行了真实 MVCNN 模型。

57.2 前置条件

validateMultiViewJobAndViews 检查任务与视角数、来源、模型/窗口等绑定；终端报告还由 coordinator 核对 job/attempt/terminalOwner。最少独立 drone 视角和单视角配对基线是不同实验输入，不应混为一次模型调用。

57.3 模型实现

IMultiViewRecognitionAlgorithm 是 C++ 扩展点；tools/mvcnn_onnx_worker.py 有真实 ONNX CPUExecutionProvider 路径。单帧 GS/ObjectDetection YOLO worker 和多视角任务、分布式 DI 请求是不同入口，默认接线需分别核对。

57.4 失败与验证

缺视角、模型/输入摘要不符、任务过期、错误 owner 与模型失败分别报告。新增算法保留输入绑定、结果字段与可复核模型身份；接口存在不证明真实硬件准确率或分布式运行资格。

完整重载、数据字段和原始注释：api/uav-reference.md。

58 API 扩展与跨模块调用检查

AC-23 · Core 接口契约

58.1 新增服务

先在应用模块定义 request/response/event 或 Fields 语义；注册 Provider handler，写明 ACK 接受条件和 Selection 后执行动作；再实现 User 调用、取消/超时与结果绑定。Core 不增加模型/航点领域字段来替代应用 payload。

58.2 新增策略或存储后端

实现对应虚接口，显式声明能力和不支持项；确保运行入口真正选择该实现。DI strategy 必须消费认证 ACK/offer，Repo backend 必须保留 artifact 身份和提交阶段，UAV algorithm 必须核对 job 与视角。

58.3 开发者核对顺序

查签名/默认值 → 查参数与数据结构 → 查触发点/前置条件 → 查 owner 的状态迁移与失败出口 → 查回调/资源回收 → 查调用方 → 执行与变更相称的验证 → 同步 Spec、设计及 Git。

58.4 文档管理

执行 MANAGEMENT.md 与 AGENTS.md 的同步要求。API 变更不仅是改函数名；默认值、线程承诺、权限门、状态和错误语义改变也必须登记。当前/目标必须独立维护，完整声明参考更新后人工审查差异。

完整重载、数据字段和原始注释：`api/core-reference.md`。